



POLITECNICO DI TORINO

MASTER'S DEGREE IN CYBERSECURITY ENGINEERING

Design and Implementation of an eBPF-based Runtime Security Monitoring Tool

Supervisor:

Prof. Riccardo Sisto

Candidate:

Simone Romano

Company Tutors:

Dr. David Soldani

Dr. Vincenzo Cosi

Academic Year 2025-2026

This thesis describes an eBPF-based tool for monitoring process activity on Rocky Linux 8.10. Its development centred on a set of kernel hooks that observe how processes are created, replace their executable image, change credentials, interact with other processes and access files, memory, namespaces and cgroups.

The hooks are attached to syscalls, tracepoints, kernel functions and Linux Security Module interfaces. Since each attachment point exposes a different stage of an operation, the resulting events distinguish between a process request, a security check, a committed state change and the result returned by the kernel. Events are encoded in a bounded binary format, sent through a perf-event channel and decoded by a Go application.

In user space, the tool loads only the probes required by the selected configuration and evaluates the decoded events through policies and deterministic detectors. Policies define the monitoring scope, whereas detectors recognise either individual events or short, ordered sequences of related activity. Alerts retain the evidence that produced them and can be classified using MITRE ATT&CK metadata.

The prototype was tested on a single-node Minikube environment using an adapted Capture the Flag challenge. During the experiment, it observed process activity originating from a separate container and reconstructed the sequence associated with shell execution and access to a protected file. These results show that eBPF hooks can support detailed, configurable process monitoring while keeping collection close to the kernel events being observed.

Acknowledgements

Only in the printed version

Contents

Acronyms	IX
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement	2
1.3 Objectives and Thesis Structure	4
2 eBPF Architecture	6
2.1 Linux Kernel Observability	6
2.2 eBPF Fundamentals	10
2.2.1 Execution Model, Verifier and JIT Compilation	10
2.2.2 Object Files, Loading and Attachment Lifecycle	12
2.2.3 Hook Selection and Event Semantics	13
2.2.4 Maps, Helpers and Kernel-to-Userspace Transport	14
2.2.5 BTF and CO-RE Portability	16
2.3 Related eBPF Runtime Security Tools	17
2.3.1 Tracee	17
2.3.2 Other Relevant eBPF Security Tools	19
2.3.3 Comparative Summary	20
2.4 Positioning of the Proposed Work	21

3	Requirements and System Design	22
3.1	Design Goals and Constraints	23
3.1.1	Target Environment and Compatibility Assumptions .	23
3.1.2	Functional Requirements	24
3.1.3	Non-Functional and Operational Requirements	25
3.2	System Architecture	25
3.2.1	Kernel-Space and User-Space Responsibilities	26
3.2.2	End-to-End Event Lifecycle	27
3.3	Kernel-Space Collection Design	28
3.3.1	Hook Selection and Logical Events	28
3.3.2	Early Event Selection and Kernel-Side Filtering	29
3.4	Event Contract and Transport Design	30
3.4.1	Stable Binary Event Contract	31
3.4.2	Perf-Buffer Transport and Failure Boundaries	32
3.5	User-Space Runtime Design	33
3.5.1	Loading, Attachment and Runtime Lifecycle	33
3.5.2	Decoding, Normalisation and Event Registration . . .	34
3.5.3	Event Output, Alert Output and Operational Logging	34
3.6	Policy and Detection Architecture	35
3.6.1	Policy-Based Event Scope	35
3.6.2	Point and Collective Detectors	35
3.6.3	Bounded Local Correlation	36
3.6.4	Alert Evidence and MITRE ATT&CK Metadata . . .	36
3.7	Design Decisions and Scope Boundaries	37
4	Kernel-Space Event Collection Implementation	38
4.1	Build Toolchain and Project Organisation	38
4.2	eBPF Program Implementation	39
4.2.1	Shared Event-Construction Framework	39

CONTENTS

4.2.2	Process Lifecycle and Executable Transitions	42
4.2.2.1	Process Creation and Lifecycle	42
4.2.2.2	Executable Transitions	44
4.2.3	Identity, Credentials, Process Control and Limits . . .	46
4.2.3.1	Identity and Credentials	46
4.2.3.2	Process Control and Limits	49
4.2.4	File and Filesystem Security	51
4.2.5	Memory, Namespace and Cgroup Transitions	55
4.2.6	Kernel Integrity and Kernel-Facing Activity	58
4.3	Binary Event Contract and Kernel Transport	61
4.4	Kernel-Side UID Filtering	62
4.5	Compatibility and Verifier Challenges	63
4.6	Kernel-Space Implementation Summary	63
5	User-Space Runtime and Detection Implementation	65
5.1	Command-Line Configuration and Runtime Bootstrap	65
5.2	Loader, Probe Registry and Attachment Lifecycle	67
5.3	User-Space Decoder and Event Registry	68
5.4	User-Space Event Selection and Admission	70
5.5	Policy and Detector Engine	71
5.5.1	YAML Policy and Detector Definitions	71
5.5.2	Point and Collective Evaluation Paths	73
5.5.3	Compiled Correlation State and Alert Deduplication .	74
5.6	Output and Structured Logging	77
5.7	User-Space Implementation Summary	78
6	Experimental Evaluation	79
6.1	Evaluation Objectives and Experimental Protocol	79
6.2	Experimental Testbed	80

CONTENTS

6.3	Controlled AES-beta Scenario	83
6.4	Functional Evaluation	84
6.4.1	Cross-Container Event Visibility	84
6.4.2	Collective Detector Validation	84
6.4.3	Benign Control and Repeated Runs	85
6.5	Performance Evaluation	86
6.5.1	Metrics, Sampling and Benchmark Profiles	86
6.5.2	Resource Consumption and Workload Duration	86
6.5.3	Observed Difference with Kernel-Side UID Filtering	87
6.6	Discussion of Results	88
6.7	Threats to Validity and Current Limitations	89
6.8	Evaluation Summary	90
A	Appendix A	91
B	Appendix B	92
	Bibliography	93

List of Figures

2.1	High-level eBPF execution workflow	11
2.2	Complementary evidence exposed by different observation points along a kernel operation.	14
2.3	Tracee high-level architecture	18
3.1	Kernel space and userspace architecture	26
3.2	Responsibility boundary and end-to-end flow of the proposed monitoring system.	27
3.3	Conceptual relationship between the binary event record, the user-space event registry, and the resulting normalised event.	31
3.4	User-space runtime lifecycle and failure boundaries	33
4.1	Execution monitoring: attempts and outcomes	44
4.2	Real, Effective, Saved and Filesystem UIDs	47
4.3	Memory and isolation transition monitoring	55
5.1	Binary record layout consumed by the user-space decoder	68
6.1	Minikube experimental testbed	81

List of Tables

2.1	Observation sources for eBPF runtime monitoring	9
2.2	Comparison of eBPF event-transport mechanisms	16
2.3	Architectural summary of the runtime security tools considered in this chapter.	20
3.1	Functional requirements of the proposed system.	24
3.2	Non-functional and operational requirements.	25
3.3	Distinction between physical hooks, producer and support probes, and logical events.	29
5.1	Runtime bounds for collective state and alert deduplication. .	76
5.2	User-space output and diagnostic boundaries.	78
6.1	Experimental environment and controlled configuration. . . .	82
6.2	Results of the repeated functional windows. Every exploit alert contained four ordered evidence events.	85
6.3	Replicated performance results, with five runs per profile. CPU and duration values summarise per-run observations; memory and RSS are the maxima across all selected samples and replicas. Metrics belong to the container named in the first column.	87

Acronyms

eBPF	Extended Berkeley Packet Filter
CPU	Central processing unit
MITRE ATT&CK	MITRE Adversarial Tactics, Techniques, and Common Knowledge
LSM	Linux Security Module
JIT	Just-in-time
BTF	BPF Type Format
CO-RE	Compile Once - Run Everywhere
ELF	Executable and Linkable Format
API	Application programming interface
CLI	Command-line interface
CGO	C language interoperability mechanism for Go
JSON	JavaScript Object Notation
YAML	Yet Another Markup Language
PID	Process identifier
TID	Thread identifier
UID	User identifier

Acronyms

GID	Group identifier
VMA	Virtual memory area
IPC	Inter-process communication
UTS	UNIX time-sharing system
KABI	Kernel application binary interface
RHEL	Red Hat Enterprise Linux
BPF	Berkeley Packet Filter
CTF	Capture the Flag
RSS	Resident set size
SSH	Secure Shell
UTC	Coordinated Universal Time
SD	Standard deviation
RAM	Random-access memory
TOTP	Time-based one-time password

Chapter 1

Introduction

1.1 Context and Motivation

Modern Linux systems execute applications, containers, and infrastructure services whose security-relevant behaviour changes continuously at runtime. Static inspection can describe software before execution, while application logs expose only the information that applications choose to report. Neither source alone provides a complete account of operations such as process creation, credential changes, file access, memory protection changes, namespace activity, or interactions with kernel security mechanisms. Runtime security monitoring therefore requires an observation point that is close enough to the operating system to capture these actions as they occur.

The Linux kernel is a valuable observation layer because it mediates processes, files, credentials, memory, and other shared resources. Kernel-level telemetry can reveal behaviour that is distributed across several applications or omitted from their logs. This visibility is particularly relevant in containerised environments, where workloads remain isolated at the application level but share the host kernel. At the same time, kernel instrumentation is security-sensitive: excessive privileges or unsafe extensions may increase the attack surface, especially across container boundaries [21]. A monitoring design must consequently balance visibility, compatibility, operational cost, and the risks associated with privileged execution.

Runtime security tools increasingly use eBPF to obtain this visibility. Existing systems demonstrate that eBPF can support process, file, network, and security telemetry in cloud-native environments, although their archi-

textures and performance characteristics differ [22]. This thesis focuses on the process and host-security side of that design space.

eBPF provides a controlled runtime inside the Linux kernel. User-space software loads constrained programs that can be attached to supported processes, networking, and security hooks. Before a program is accepted, the kernel verifier analyses its control flow and use of memory, registers, pointers, and helper functions through static analysis. Accepted programs can then collect or transform data when the corresponding hook is reached [19, 23, 24].

This execution model is well suited to runtime monitoring for three reasons. First, collection occurs at the point where the kernel processes the operation, reducing dependence on application-specific instrumentation. Second, eBPF maps and event buffers provide structured communication between kernel and user space. Third, programs can reject irrelevant activity before transporting it to user space, which creates an opportunity to control processing overhead. These properties do not make an eBPF tool automatically safe, portable, or efficient: verifier behaviour, available program types, helper functions, kernel data structures, and attachment mechanisms vary across kernel versions. They instead provide the foundation on which a target-aware monitoring pipeline can be built.

The work was developed in an industrial collaboration and targets Rocky Linux 8.10 with the kernel `4.18.0-553.137.1.el8_10.x86_64`. This environment is important because it combines a long-lived enterprise kernel line with backported functionality. A feature exposed by a newer upstream kernel cannot be assumed to exist, or to behave identically, on this target [28]. Compatibility is therefore treated as an architectural constraint.

Within this target environment, the prototype is designed as a host-level runtime monitor. It collects and processes security-relevant events on the monitored system and performs bounded local detection over individual events and short event sequences.

1.2 Problem Statement

The central problem addressed by this thesis is how to convert heterogeneous kernel observations into useful and readable security information on a constrained enterprise Linux target. Capturing an event is only the first step. A practical pipeline must assign a stable identity to the event, transport it

without confusing records produced by different probes, decode typed arguments, apply user-selected scope, and present the result consistently. When these contracts diverge, valid kernel records may be decoded as unknown events, attributed to the wrong event type, or silently discarded.

Individual observations also have different analytical value. Some are relevant in isolation, for example an access to a sensitive file under narrowly defined conditions. Others become relevant only as an ordered group, such as a credential transition followed shortly by process execution or sensitive file access. Following the established anomaly-detection taxonomy, the thesis uses *point event* for a locally significant observation and *short collective event pattern* for a bounded sequence whose combined meaning is more relevant than its elements considered independently [15]. These terms describe deterministic, rule-based detection; they do not imply machine learning, statistical baselines, or unsupervised anomaly detection methods.

Collecting kernel events alone does not make the resulting telemetry immediately useful for security analysis. The proposed system therefore introduces a configurable policy and detection layer that transforms selected runtime observations into focused security information. Policies define the monitoring scope by selecting the event domain relevant to a particular scenario, helping to limit noise and unnecessary processing. Detectors operate on the resulting normalised event stream and encode reusable conditions that can generate an alert from either an individual event or a short, locally correlated sequence. This separation allows collection scope and detection logic to evolve independently, without requiring each analytical change to be implemented inside the eBPF programs. The generated alerts are enriched with metadata based on MITRE ATT&CK [29], providing a standard vocabulary for interpreting the observed behaviour and supporting downstream analysis.

Four constraints shape the problem. First, the target kernel limits which attachment mechanisms, helpers, maps, and portability techniques can be used reliably. Second, the kernel verifier constrains program complexity and memory access, so event enrichment must be divided carefully between kernel and user space. Third, high-frequency hooks can generate enough data to make transport, decoding, rule evaluation, and output formatting more expensive than the probe itself. Fourth, the agent is expected to run beside production workloads; CPU and memory use are therefore evaluation criteria rather than secondary implementation details.

These constraints motivate a selective architecture. The proposed tool uses

eBPF for collection and optional early filtering, while user space owns event normalisation, policy management, detector evaluation, alert generation, and presentation. The user-space runtime is implemented in Go, which was selected in part because libraries such as `libbpfgo` provide Go bindings to `libbpf` for loading eBPF objects and managing programs, maps, and event buffers [3]. The design preserves a stable kernel-to-user-space contract while allowing detection logic to evolve without recompiling every eBPF program.

1.3 Objectives and Thesis Structure

The main objective of this thesis is to design, implement, and evaluate an eBPF-based runtime security monitoring prototype that collects process and security-relevant kernel events on the target Rocky Linux environment, normalises them in user space, and evaluates them through configurable policies and local detectors.

The remainder of the thesis develops this objective progressively, moving from the necessary technical foundations to the architecture, implementation, and evaluation of the proposed system:

- **Chapter 2 - eBPF Architecture** introduces Linux runtime instrumentation, the eBPF execution model, object loading and attachment, maps, transport, and portability. It also reviews related eBPF-based runtime-security tools.
- **Chapter 3 - Requirements and System Design** defines the target environment, functional and operational requirements, architectural components, event flow, and the boundaries between kernel-space collection and user-space processing.
- **Chapter 4 - Kernel-Space Event Collection Implementation** describes the shared eBPF event framework, process and security hook families, binary record contract, perf-buffer transport, early filtering, and compatibility work required by the target kernel.
- **Chapter 5 - User-Space and Detection Implementation** presents the Go runtime, probe registry, attachment lifecycle, decoder, event selection, policy and detector subsystems, MITRE ATT&CK metadata, output handling, structured logging, and container packaging.

- **Chapter 6 - Experimental Evaluation** evaluates the prototype in a Kubernetes environment deployed with Minikube. The experimental scenario uses a vulnerable CTF workload as the target of controlled attacks to assess event collection, detector operation and alert generation, together with the prototype's CPU and memory overhead.
- **Chapter 7 - Conclusions** summarises the results, discusses the principal limitations of the prototype, and identifies possible directions for future development.

Chapter 2

eBPF Architecture

2.1 Linux Kernel Observability

Runtime monitoring requires evidence about operations while they are being executed. Application logs describe domain-specific actions, but their content depends on what each application chooses to record. The Linux kernel offers a complementary observation layer because it mediates process creation, credential changes, file access, memory management, inter-process communication, and other security-relevant operations. Kernel observability is therefore useful when behaviour crosses application boundaries or is not represented consistently in application logs [52, 19].

Linux does not expose all runtime activity through one universal event source. Instead, it provides instrumentation points at different layers of kernel execution. Each mechanism determines when monitoring code runs, which context is available, and how strongly the observation depends on kernel internals.

System-call observation. A system call is the controlled interface through which a user-space process requests a service from the kernel. It is not itself an instrumentation mechanism. Monitoring systems commonly observe this boundary through system-call tracepoints, such as entry and exit events. An entry event exposes the requested operation and its original arguments, whereas an exit event exposes the returned result. Correlating the two can distinguish an attempted operation from one that completed successfully. This view is broad and process-oriented, but it remains close to the user-space request: one system call may traverse several internal functions, and

its original arguments may not identify the final kernel object or security decision [57, 52].

Tracepoints and raw tracepoints. A tracepoint is a static observation point deliberately inserted into the kernel source by kernel developers. It has a name, belongs to an event category, and defines the data made available when that point is reached. A tracepoint remains inactive until a tracing component is attached to it. In an eBPF-based monitor, this component is an eBPF program loaded from user space and attached to the selected tracepoint. Each time kernel execution reaches that point, the kernel invokes the attached program and provides a tracepoint-specific context containing the declared event fields.

A regular tracepoint gives the eBPF program a structured event context whose layout follows the format exported for that tracepoint. A raw tracepoint refers to the same underlying kernel instrumentation site, but exposes the original arguments passed by the kernel to that site with less intermediate transformation. This reduces preparation performed before the eBPF program runs, while requiring the program to know how each raw argument must be interpreted. Tracepoints are therefore explicit and comparatively stable observation points, whereas raw tracepoints trade some of that abstraction for more direct access to the underlying arguments. Both mechanisms are limited to the tracepoints actually provided by the target kernel [57, 55].

Kprobes and kretprobes. A kprobe is a dynamic probe placed at a selected kernel instruction at run time. Monitoring tools usually identify that instruction through the symbol of a kernel function and place the probe at its entry. Unlike a tracepoint, the observation point does not need to have been prepared explicitly by the kernel developer. When execution reaches the probed instruction, the kernel temporarily transfers control to the probing infrastructure, which invokes the attached eBPF program before normal execution continues. The program receives a snapshot of the processor-register state. Function arguments are not presented as a tracepoint-specific event structure; they must be recovered from the appropriate registers according to the architecture and calling convention.

A kretprobe observes the corresponding function-return path. It causes an attached program to run when an invocation of the probed function finishes and makes the return value available through the register context. A kprobe can therefore describe the operation requested at function entry, while a

kretprobe can reveal the result produced at function exit. Entry and return records are separate observations: when both the original arguments and the result are required in one logical event, the monitoring system must save the entry data and correlate it with the return observation, commonly by using the current thread identity.

Kprobes and kretprobes provide access to functions for which no tracepoint exists, but this flexibility increases coupling to kernel implementation details. A renamed or inlined function, a changed signature, a different calling convention, or an unavailable symbol can invalidate the attachment or the argument-decoding logic. They are therefore powerful dynamic mechanisms, but generally require more target-specific validation than tracepoints [49].

Linux Security Module hooks. The LSM framework inserts security mediation points into kernel operations involving files, credentials, processes, sockets, and other protected objects. At these points, registered security modules can inspect an operation and, for hooks with authorisation semantics, permit or reject it. Observing an LSM-related function can therefore expose a resolved kernel object and the security-relevant stage of an operation, rather than only the original system-call request. Two concepts should not be confused: an LSM hook is the kernel mediation point, while an eBPF LSM program is one possible program type for attaching logic to supported hooks. On kernels where that program type is unavailable or unsuitable, monitoring tools may instead observe the corresponding security function through a kprobe. An entry-side observation still does not necessarily reveal the final outcome; when the result matters, it must be obtained from a compatible return-side observation [51, 55, 49].

Observation source	Information exposed	Main trade-off
System-call tracepoint	Arguments at entry or return value at exit.	Broad process visibility, but limited insight into internal kernel processing and resolved objects.
Kernel tracepoint	Structured fields defined for a specific kernel event.	Clear semantics and simple decoding, but only where a suitable tracepoint is available.
Raw tracepoint	Original tracepoint arguments with minimal preprocessing.	More direct access, but requires kernel-aware argument decoding.
Kprobe / kretprobe	Register state at function entry or return.	Flexible placement, but stronger dependence on kernel symbols, signatures and calling conventions.
LSM-related hook	Security context, resolved objects and mediation stage.	Rich security semantics, but attachment and result visibility depend on target-kernel support.

Table 2.1: Comparison of the principal observation sources relevant to eBPF-based runtime monitoring [57, 49, 51, 55].

The role of eBPF. eBPF is the execution facility attached to an observation point, not a replacement name for the observation point itself. The selected hook causes an eBPF program to run and determines the context received by that program. The program can then extract selected fields, apply an early filter, update maps, or submit an event to user space. Consequently, “tracepoint”, “kprobe”, “LSM hook”, and “eBPF program” describe different parts of the same monitoring path [23, 55].

These mechanisms are complementary. System-call observations describe requests and results at the user–kernel boundary; general tracepoints expose kernel-defined events; kprobes and kretprobes provide dynamic access to function entry and return; and LSM-related hooks expose security mediation points. A runtime monitor combines them according to the required semantics, available target-kernel features, and acceptable coupling to kernel internals. The engineering objective is to select observation points that provide sufficient and reliable evidence for the intended security analysis [52, 22].

2.2 eBPF Fundamentals

eBPF is a modern technology that makes the Linux kernel programmable at runtime. It allows small programs to be loaded from user space and executed inside the kernel when a selected event occurs. Before execution, the kernel verifies that each program respects the safety constraints associated with its program type and intended attachment point [30, 23].

This approach makes it possible to extend kernel behaviour without modifying the kernel source or developing a traditional kernel module for each new functionality. eBPF programs can be loaded, replaced, and specialised for different purposes, including system observability, performance analysis, network processing, and security. Its principal benefit is therefore controlled programmability: functionality can be introduced close to the kernel operations of interest while remaining subject to verification and well-defined execution constraints [19, 24].

2.2.1 Execution Model, Verifier and JIT Compilation

An eBPF program is compiled into instructions for the BPF virtual machine and submitted to the kernel through the BPF user-space interface. The instruction set defines general-purpose registers, a bounded stack, arithmetic and branch instructions, memory operations, and calls to kernel-provided functions. The program does not execute as unrestricted kernel code: its available context, memory accesses, and callable functions depend on its program type [30, 25].

Because the programs described above are loaded from user space but execute inside the kernel, they cannot be allowed to run without first establishing that they respect the kernel’s safety constraints. An unsafe memory access or an invalid operation in this context could affect the stability and security of the entire system. Linux addresses this requirement through the eBPF verifier, the kernel component responsible for deciding whether a submitted program can be accepted for execution. Rather than trusting the program, the verifier analyses it during the loading process and before permitting its execution; a program that does not satisfy the required safety conditions is rejected and never executed.

The verifier performs a path-sensitive static analysis of the program’s control flow. It follows the possible execution paths while tracking the state of registers and stack locations, distinguishing scalar values from different

classes of pointers, checking memory bounds and alignment, and validating the arguments supplied to helper functions. Paths that may access invalid memory, use uninitialised values, or leave kernel-managed resources unreleased are rejected. To avoid analysing equivalent situations repeatedly, the verifier uses state pruning: when a previously accepted verifier state already covers a new state reached at the same instruction, the redundant path does not need to be explored again [24, 20].

Verification is a necessary safety gate, not a proof that a program implements the intended monitoring semantics. A verifier-accepted program may still read the wrong field, emit an incomplete event, or encode data inconsistently with its user-space decoder. Conversely, semantically reasonable source code may be rejected because the verifier cannot establish the required safety properties for a particular kernel. Verifier behaviour is therefore both a security mechanism and a practical compatibility constraint [24, 40].

Once the verifier accepts a program, the kernel may pass its eBPF instructions to an architecture-specific JIT compiler. The compiler translates the verified eBPF bytecode into native machine instructions for the host architecture. The resulting native code can then be executed directly whenever the program is invoked, avoiding the overhead of processing the eBPF instructions individually at each execution [46, 48].

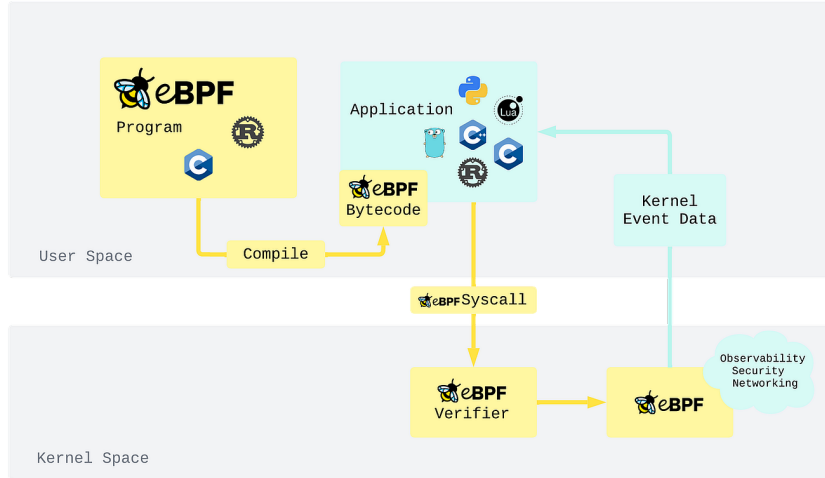


Figure 2.1: High-level eBPF execution workflow.

JIT compilation does not alter the functionality authorised by the verifier.

The translation is performed only after verification and preserves the semantics of the accepted eBPF program. Its availability depends on the host architecture, the kernel build configuration, and the runtime JIT settings.

This completes the basic eBPF execution path considered here. A program is compiled into eBPF instructions, submitted to the kernel, analysed by the verifier and, if accepted, translated into native machine code and executed whenever its associated hook or execution context is invoked [19].

2.2.2 Object Files, Loading and Attachment Lifecycle

The source code of an eBPF application is not passed directly to the kernel. Clang first compiles it into BPF instructions stored in an ELF object. The object groups programs into named sections and also carries map declarations, licence information, symbols, BTF metadata and relocation records. Section names help a loader infer the program type and expected attachment family, while BTF and relocation information describe references that must be adapted before the program can be accepted by the target kernel [55, 45, 41].

A user-space loader, commonly built on `libbpf`, opens this object and turns its declarations into kernel objects. It creates or reuses the required maps, resolves relocations and submits each enabled program through the BPF system-call interface. The kernel then applies the verifier to every submitted program. Loading is therefore an active compatibility boundary: a syntactically valid object may still fail because a required program type, helper, kernel symbol or verifier behaviour is unavailable on the host [26, 50, 24].

Loading and attachment are separate operations. A successfully loaded program exists as a kernel object referenced from user space, but it does not run until it is connected to a compatible hook. Depending on the program and hook type, this connection may be represented by a BPF link or by tracing-specific infrastructure such as a perf event. The attachment owns the relationship between the program and its trigger and can be removed independently from the program itself. This separation allows one object file to contain several programs, only a subset of which may be loaded and attached for a particular monitoring configuration [55, 50].

File descriptors provide the user-space references through which programs, maps and links are configured and eventually released. Closing the final reference normally permits the corresponding kernel object to be destroyed,

whereas pinning can give an object a persistent name in the BPF filesystem and allow it to outlive the process that created it. Runtime ownership and cleanup are consequently part of the operational model, not merely loader housekeeping: stale attachments or pinned objects may continue consuming kernel resources after the original control path has ended [26, 50].

2.2.3 Hook Selection and Event Semantics

An eBPF program type defines the execution contract enforced by the kernel, including the expected context, available helpers, and permitted return semantics. The hook instead identifies the event or kernel execution point that triggers the program, while the attachment mechanism binds the loaded program to a compatible hook. These concepts are related, but they are not interchangeable [55, 25].

The selected observation point determines not only when an eBPF program runs, but also the meaning of the resulting event. A system-call entry hook captures the operation requested by a process and its original arguments. A later tracepoint, dynamic probe, or LSM-related hook may expose kernel-resolved objects or the security mediation stage of the operation. An exit or return-side observation can instead reveal whether the operation completed successfully [57, 49, 51].

These observations represent different stages of the same kernel execution path and should therefore be considered complementary rather than interchangeable. No single hook necessarily exposes the original request, the resolved kernel context, and the final result. A runtime monitor may consequently combine multiple event sources and correlate their records through process identity and temporal proximity.

Hook selection also introduces a compatibility trade-off. Explicit tracepoints provide structured interfaces when the target kernel exports a suitable event, whereas dynamic probes offer broader placement flexibility but depend more strongly on kernel symbols, function signatures, and calling conventions. The appropriate observation point must therefore balance the required security semantics with the capabilities and stability of the deployment kernel [57, 49, 28].

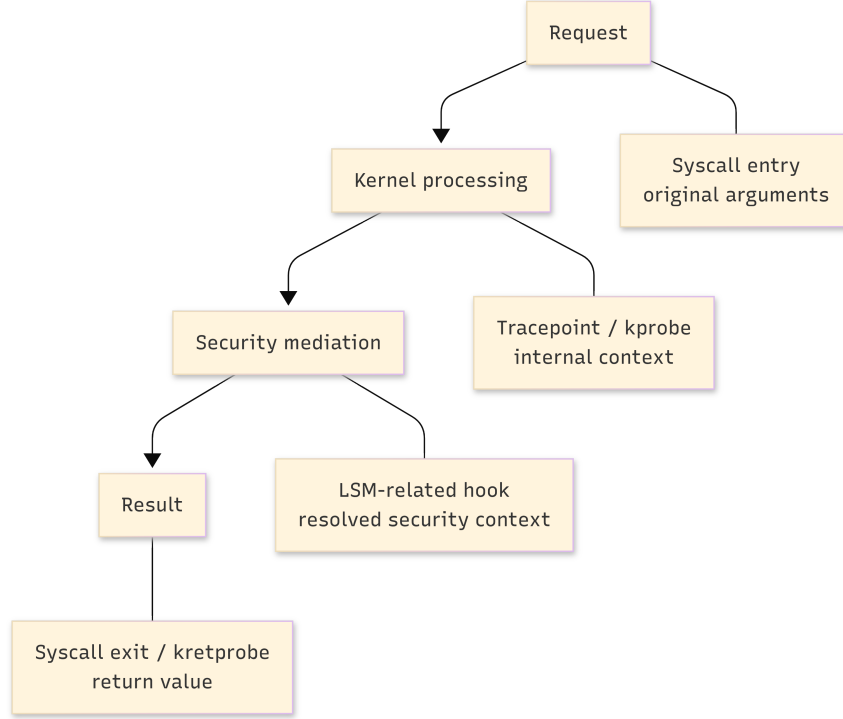


Figure 2.2: Complementary evidence exposed by different observation points along a kernel operation.

2.2.4 Maps, Helpers and Kernel-to-Userspace Transport

The execution of an eBPF program is intentionally constrained and usually short-lived. Runtime monitoring, however, requires information to persist beyond a single invocation and eventually reach user space, where more complex processing can be performed. Linux supports this interaction through kernel-managed state and a controlled communication path between eBPF programs and the user-space runtime. Maps, helpers, and event buffers form the main building blocks of this path [42, 25, 54, 43].

Maps are typed kernel objects that allow eBPF programs to preserve state across invocations and exchange data with user space. Each map type defines its own access and concurrency semantics, while its structure and capacity are established at creation time. By moving persistent configuration and temporary state outside the constrained BPF stack, maps extend the

information that a program can retain between executions. Accesses performed by eBPF code remain subject to verifier validation and to the rules imposed by the selected map type [42, 26].

Different map organisations serve different ownership patterns. Array maps provide fixed indexed storage and are suitable for configuration or bounded workspaces. Hash maps associate dynamically selected keys with values and can retain state such as entry arguments until a later hook completes an operation. Per-CPU variants allocate an independent value for each processor, reducing contention when programs update temporary state or counters on a hot path. Their values must, however, be combined explicitly when user space needs a system-wide result. Shared maps instead require the application to consider concurrent access from several CPUs and the atomicity of each update [42].

Maps also form a control channel in the opposite direction. User space can populate configuration, selection or lookup data before monitoring begins and update supported values while programs are loaded. An eBPF program can then consult that state without embedding every operational choice in its instructions. This division is central to configurable monitoring: the kernel program remains bounded, while user space controls which state it exposes and how map contents are interpreted [42, 26].

Helpers are kernel-provided functions that extend what an eBPF program can do without granting unrestricted access to kernel internals. They expose selected services through a controlled interface, allowing the kernel to validate how those services are invoked and what data is passed to them. The set of available helpers is determined by the program type and by the capabilities of the target kernel, so support documented for eBPF in general does not imply that the same helper can be used in every execution context. This constraint is particularly relevant to this work, which targets Rocky Linux 8.10 with kernel 4.18.0-553.137.1.el8_10.x86_64: helper selection must therefore be validated against the capabilities of the deployed kernel and the program type used by each probe, rather than inferred solely from current upstream documentation [25, 24].

Event transport carries records produced during eBPF execution to user space, where they can be decoded and processed without extending the work performed inside the kernel. The BPF ring buffer provides a shared multi-producer, single-consumer channel designed to use buffer capacity across CPUs and preserve the order in which records are reserved. Perf-event transport follows a different model: a `BPF_MAP_TYPE_PERF_EVENT_ARRAY` directs

records to per-CPU perf ring buffers, which are consumed independently and can report lost events.

Both mechanisms rely on finite buffer capacity. When kernel-side producers generate data faster than user space can consume it, a submission may fail or a record may be lost. Event transport must therefore be treated as part of the monitor’s correctness model rather than as a transparent delivery channel [43, 54, 25].

Mechanism	Buffer organisation	Main implication
BPF ring buffer	Shared multi-producer, single-consumer buffer	Shares capacity across CPUs and preserves reservation order between producers.
Perf-event transport	Per-CPU buffers referenced through a perf-event array	Buffers are consumed independently and provide explicit lost-event notifications.

Table 2.2: Comparison of eBPF event-transport mechanisms [43, 54].

Regardless of the selected transport mechanism, kernel and user space must share a consistent record format so that the transferred data can be decoded into correctly typed events [19].

2.2.5 BTF and CO-RE Portability

An eBPF program often interacts with kernel data structures whose layout may change across kernel versions and distribution builds. A program compiled against one specific set of kernel headers may therefore rely on field offsets or type definitions that do not match the target system. Portability requires a way to preserve the meaning of those accesses while adapting them to the types exposed by the deployed kernel. BTF and CO-RE provide the metadata and relocation mechanisms used to address this problem.

BTF represents types and related metadata in a compact format understood by the kernel and BPF tooling. BTF may describe C types and strings and can also carry function, line, and relocation information. When kernel BTF is available, loaders and debugging tools can reason about kernel data structures without depending solely on a set of headers copied from one kernel build [45].

CO-RE uses BTF information to adapt an eBPF program to the kernel

on which it is loaded. During compilation, Clang stores relocation records that describe the kernel types and fields accessed by the program. At load time, `libbpf` reads the BTF information of the running kernel and updates the relevant eBPF instructions so that they use the actual layout of those structures. As a result, the same compiled object can continue to work when compatible kernel versions place a field at a different offset, without requiring the program to be recompiled on the monitored system [45, 41, 50].

In this thesis, this mechanism helps the eBPF programs access kernel structures according to the layout exposed by Rocky Linux 8.10 with kernel `4.18.0-553.137.1.el8_10.x86_64`. However, CO-RE only adapts compatible differences in type and data-structure layout. It cannot make a missing hook, helper, program type, or attachment mechanism available, and it does not ensure that the target verifier will accept the program. These capabilities must therefore be checked directly on the deployed kernel. This is especially important for enterprise distributions, where backported features may make the effective kernel capabilities differ from those suggested by the upstream version number alone [40, 28].

2.3 Related eBPF Runtime Security Tools

Existing eBPF-based runtime security tools share the objective of interpreting kernel-derived activity, but they place event selection, detection, and response at different points of the monitoring pipeline. Tracee, Falco, and Tetragon are considered here because they represent three relevant design approaches for positioning the proposed system. Tracee provides the closest architectural reference through its separation of event collection, policies, detectors, and output. Falco centres its design on declarative rules and alert generation, while Tetragon moves filtering and optional response closer to the monitored kernel hook [22].

2.3.1 Tracee

Tracee is the closest architectural reference among the systems considered in this chapter. It separates eBPF-based collection from user-space event processing, policy selection, detection and output. Figure 2.3 shows this staged pipeline: kernel and networking hooks generate observations, which are converted into events before higher-level analysis and delivery [8, 4, 22].

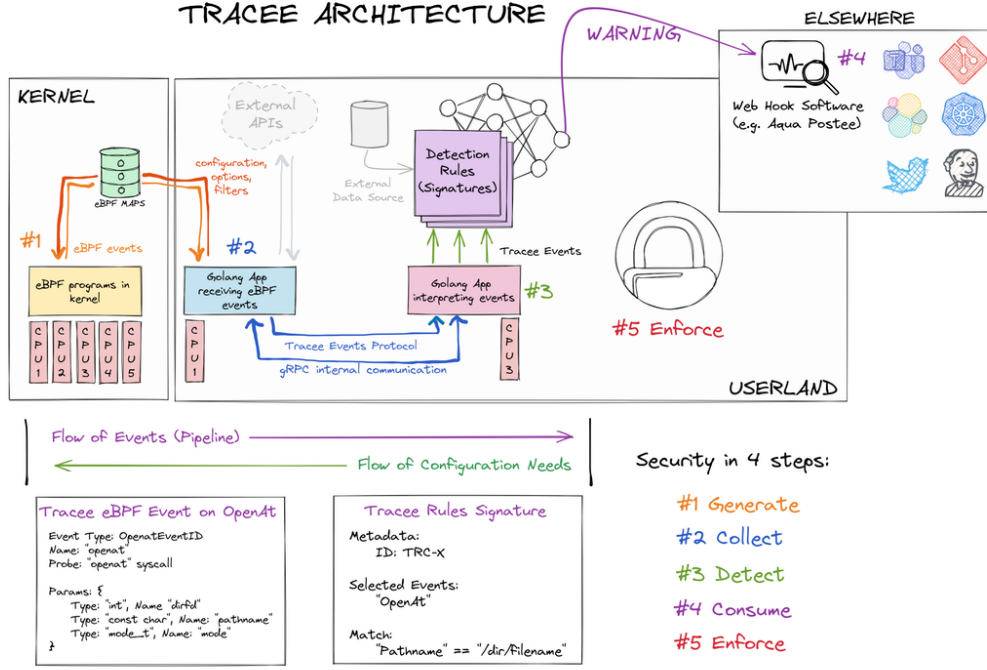


Figure 2.3: Tracee high-level architecture [4].

The figure reflects the signature-oriented terminology of the release for which it was published. Current documentation retains the same broad pipeline while using detectors as the preferred mechanism for new threat detections and derived events. Observations from heterogeneous hooks and detector results are represented through a common event abstraction, allowing policies, detectors and output components to remain independent from physical attachment details [10, 5, 7].

Tracee policies combine workload scope with rules that select events and optionally constrain their data. Current policy rules can also select detectors through threat metadata. Detectors consume declared input events, may retain state and produce derived events with provenance. This separates monitoring scope from security interpretation without defining a general-purpose temporal language [12, 13, 6].

Output remains independent from event production and detection, supporting human-readable and structured destinations together with additional

process and container context. Tracee also provides routing and forensic capabilities that extend beyond the prototype developed in this thesis [11, 9].

Its portability and trust boundaries are equally relevant. BTF, symbols, backports and individual hook support must be verified on enterprise kernels, even where the general kernel line is supported. Tracee also assumes a trustworthy kernel at monitoring startup; a kernel-level adversary may interfere with the observation mechanism [2, 14].

The proposed system adopts the architectural separation between collection, policy selection, detection and output, but not Tracee’s breadth. It targets a selected process and host-security event domain on one Rocky Linux environment and uses deterministic point conditions and bounded local sequences. Tracee is therefore prior art and an architectural reference rather than a claim of functional equivalence.

2.3.2 Other Relevant eBPF Security Tools

Falco is a runtime security engine organised around rules evaluated over event sources. Kernel system-call activity can be captured through its driver architecture, while plugins provide additional sources. Falco rules are declarative YAML objects containing a condition, output description, priority, and optional reusable lists, macros, tags, and exceptions. A matching rule produces an alert that can be routed to configured outputs [33, 36, 34].

This model demonstrates the operational value of readable rules, reusable expressions, explicit exceptions, and an alert-oriented interface. Falco also maintains process and container context used to enrich individual events. However, its documented rule conditions are evaluated against an incoming event, and separate event sources are processed independently; the core rule language is not presented as a general temporal sequence language across heterogeneous sources [32, 35].

Tetragon combines eBPF-based observability with optional runtime enforcement. Its tracing policies specify kernel hooks, captured arguments or return values, and selectors that can match process, namespace, capability, workload, and argument attributes. Supported actions can include emitting an event, delivering a signal, or overriding a return value where the hook and kernel support that operation [62, 63, 59].

Tetragon therefore moves more selection and potential response close to

the hook than an architecture centred on user-space detectors. It preserves process-lifecycle context and emits structured events through its own interfaces, but its documented tracing-policy model is principally hook-centred rather than a general language for arbitrary temporal sequences [60, 62]. Its current installation requirements also illustrate the dependence of advanced eBPF functionality on kernel capabilities and BTF availability [61].

2.3.3 Comparative Summary

The systems examined in this section represent different approaches to runtime security. Tracee provides the closest architectural reference because it separates kernel-level collection from the analysis performed in user space. Falco instead places declarative rule evaluation and alert generation at the centre of its design, while Tetragon moves a greater part of event selection and optional response closer to the kernel hook.

These approaches are not directly interchangeable, but they provide useful reference points for the proposed system. Tracee illustrates the event-oriented pipeline that most closely influenced its architecture, whereas Falco and Tetragon establish the boundaries represented by an alert-centred rule engine and an enforcement-oriented design. The main distinctions are summarised in Table 2.3.

Tool	Core approach	Role in the comparison
Tracee	Event-oriented pipeline with separate policy, detection, and output stages.	Closest architectural reference for the proposed system. [12, 7]
Falco	Declarative rules evaluated over incoming events to generate alerts.	Reference for a mature rule- and alert-centred model. [36, 35]
Tetragon	Hook-centred policies with in-kernel selectors and optional actions.	Reference for kernel-side filtering and runtime enforcement. [62, 59]

Table 2.3: Architectural summary of the runtime security tools considered in this chapter.

2.4 Positioning of the Proposed Work

The proposed work occupies a deliberately bounded part of the Linux runtime security design space. It focuses on process and host-security monitoring on Rocky Linux 8.10 with kernel 4.18.0-553.137.1.el8_10.x86_64. Because this environment combines an older upstream kernel base with vendor backports, compatibility cannot be inferred from the version number alone. The available hooks, helpers, data layouts, and verifier behaviour must instead be established on the deployed system [28].

Within this environment, eBPF programs collect selected kernel observations and transfer them to a Go runtime through an explicit event contract. The resulting records become a typed user-space event stream. Chapter 3 defines how that stream is scoped through policies and interpreted by detectors; the present chapter is concerned with the eBPF collection model and the prior-art systems against which the complete architecture is positioned.

The individual mechanisms used by the prototype have clear precedents in existing runtime security systems. Tracee provides the closest architectural reference through its event-oriented separation of collection, policy selection, detection, and output. Falco demonstrates the maturity of declarative event rules and alert generation, while Tetragon illustrates a more kernel-centred design that also supports enforcement [12, 7, 36, 62].

The contribution of this thesis therefore lies in the independent integration of these ideas within a compact prototype tailored to the selected enterprise kernel. Particular attention is given to compatibility on that environment, complementary observation points and the contract through which kernel records become typed user-space events. Chapter 3 develops the system design built on these foundations.

Chapter 3

Requirements and System Design

The previous chapter established the technical basis for the work and positioned the proposed system within the runtime security landscape. Building on that foundation, this chapter defines the requirements that guide the system and develops the architecture chosen to satisfy them. The discussion focuses on the constraints imposed by the target environment and on the division of responsibility between kernel space and user space.

The design is guided by a simple principle: work that depends directly on the observed kernel operation should remain small and bounded, whereas higher-level interpretation should take place in user space, where it can evolve without increasing verifier-sensitive kernel logic. An explicit event contract connects the two domains. Once an observation crosses this boundary, policies determine whether it belongs to the monitored scope, while independently configured detectors evaluate individual events or short ordered sequences. These choices are presented here at the architectural level. Their kernel-space implementation is developed in Chapter 4, the user-space runtime and detection implementation in Chapter 5, and their experimental assessment in Chapter 6.

3.1 Design Goals and Constraints

The system is designed to provide focused visibility into security-relevant host activity rather than an exhaustive account of kernel execution. Its event domain therefore centres on process execution and on operations that affect the security state of the monitored host. The architecture must allow this domain to be selected according to the monitoring objective, while presenting the resulting observations through a consistent typed event model. Security interpretation should then operate on that model without requiring the detection logic to be embedded in the eBPF programs.

This design is shaped primarily by the characteristics of the target environment and by the continuous nature of runtime telemetry. Kernel capabilities must be established on the concrete enterprise system, since its reported upstream version does not fully describe the available functionality. At the same time, frequent observations require the system to avoid unnecessary collection and to limit the state retained for ongoing detections. Any loss or failure along the event pipeline must also remain observable, so that missing telemetry is not interpreted as evidence that no relevant activity occurred.

3.1.1 Target Environment and Compatibility Assumptions

The target environment is Rocky Linux 8.10 on x86-64, running the exact kernel build `4.18.0-553.137.1.el8_10.x86_64`. This target belongs to a long-lived enterprise kernel line in which selected fixes and features are backported while the reported upstream base version remains stable. The nominal version therefore cannot be used alone to infer the effective eBPF capabilities of the host; the functionality required by the monitoring system must be confirmed on the deployed kernel [28].

The architecture assumes that the monitored host exposes usable kernel BTF information and grants the privileges required to load and attach the selected programs. As discussed in Chapter 2, BTF and CO-RE can adapt compatible differences in kernel data-structure layout, but they cannot provide missing kernel functionality, bypass permissions, or guarantee verifier acceptance [45, 41, 50, 24].

This target-aware design deliberately bounds the portability claim. The system is designed around the identified Rocky Linux environment and is not assumed to operate unchanged across arbitrary distributions or kernel configurations. Dynamic probes introduce an additional dependence on the

availability of kernel symbols and compatible function signatures. Loading and attachment failures must consequently be detected before monitoring begins, rather than allowing the runtime to continue with an incomplete observation domain.

3.1.2 Functional Requirements

Table 3.1 defines the functional requirements that guide the system architecture. They describe system-level capabilities rather than individual commands or source-code components.

ID	Functional requirement
FR-01	Collect a selectable set of process and security-relevant events from the monitored host.
FR-02	Activate only the kernel observation components required to produce the selected logical events and their dependencies.
FR-03	Transfer kernel records to user space through the selected perf-event transport and convert valid records into normalised typed events according to an explicit shared contract.
FR-04	Load configurable policies that constrain the monitored event and process scope and may restrict the configured detector catalogue before runtime construction.
FR-05	Construct a separate detector engine from the definitions admitted by the active configuration and policies, and evaluate deterministic conditions over single events or bounded ordered sequences.
FR-06	Generate alerts that preserve the observations responsible for a match and propagate the available MITRE ATT&CK metadata.
FR-07	Maintain independent presentation paths for raw events, detector alerts, and operational diagnostics.
FR-08	Provide a controlled runtime lifecycle that validates configuration, reports observable failures, responds to cancellation, and releases acquired resources during shutdown.

Table 3.1: Functional requirements of the proposed system.

Together, these requirements establish the boundaries developed in the remainder of the chapter. FR-01–FR-03 concern collection and the event contract, FR-04–FR-06 define the analytical path, and FR-07–FR-08 govern presentation and runtime lifecycle.

3.1.3 Non-Functional and Operational Requirements

The requirements in Table 3.2 define the compatibility, reliability, and resource constraints under which the functional pipeline must operate.

ID	Non-functional or operational requirement
NFR-01	Operate on the identified enterprise-kernel target with explicit compatibility assumptions and validation against the deployed system.
NFR-02	Keep kernel-side execution and data extraction bounded and acceptable to the target verifier.
NFR-03	Maintain a consistent kernel-to-user-space event contract and visibly reject unknown or malformed records before further processing.
NFR-04	Keep detection deterministic, configuration-driven, and independent from event presentation.
NFR-05	Bound incomplete collective-detection state in both retention time and capacity.
NFR-06	Report setup failures, transport loss, decoding errors, and detector failures through dedicated diagnostic channels, separate from monitored events and generated alerts.
NFR-07	Assess whether steady-state user-space processing remains below 5% of one CPU core for explicitly defined representative profiles.

Table 3.2: Non-functional and operational requirements.

NFR-07 defines an evaluation objective rather than an unconditional performance guarantee. Its assessment therefore depends on the monitored workload, enabled events, selected detectors, output configuration, and measurement method. Chapter 6 defines the representative profiles and procedures used for this evaluation.

3.2 System Architecture

The architecture separates a control path from a data path. The control path prepares the runtime by validating the configuration and rule definitions, determining which events and detectors are admitted, and resolving each logical event to the kernel programs required for its observation. The data path starts when one of these programs is triggered and carries the resulting observation through transport and user-space processing until it is presented as an event or, when a detector condition is satisfied, as an alert. Operational

diagnostics accompany both paths but remain separate from the monitoring data they describe.

3.2.1 Kernel-Space and User-Space Responsibilities

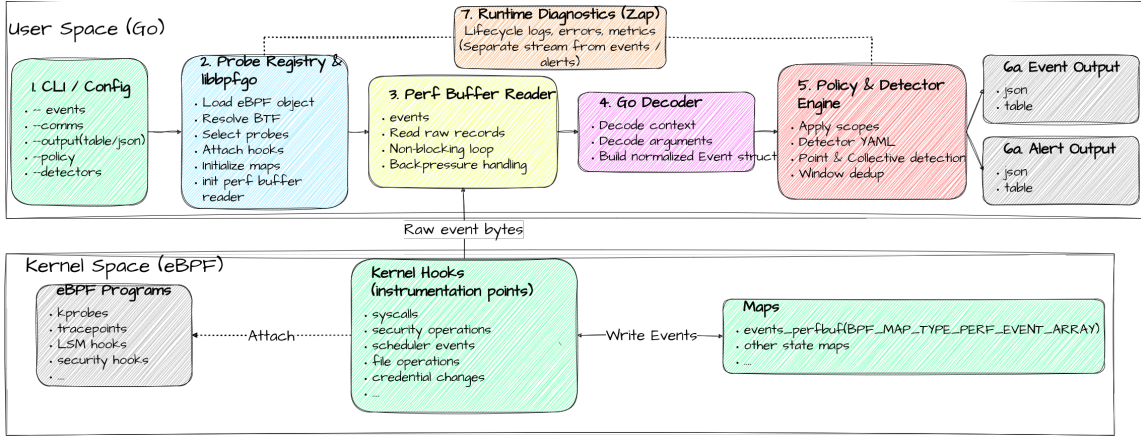


Figure 3.1: Kernel-space and user-space responsibilities.

Figure 3.1 summarises the division of responsibilities between kernel space and user space, together with the control, monitoring, and diagnostic interactions that cross this boundary.

Kernel space owns bounded observation and record construction. Selected eBPF programs collect process context and operation-specific evidence, may retain short-lived state needed to join an entry point with its return, and submit a record through the perf-event channel. The optional exact-match effective UID filter can reject records before this transfer. These mechanisms reduce traffic or complete an observation; they do not evaluate policies or classify security behaviour.

User space owns configuration and the components expected to evolve more frequently. The Go runtime determines the effective event scope, translates logical events into the required probe set, manages loading and attachment through `libbpfgo`, decodes admitted records and evaluates policies and detectors [3]. Event presentation, alert delivery and structured operational diagnostics remain separate outputs because they serve different consumers.

This boundary keeps kernel work constrained while allowing the interpre-

tation layer to change without rebuilding every collection program. It also defines a clear failure boundary: user space cannot recover an observation rejected in kernel space or lost before decoding, and kernel space does not possess the state or context required for higher-level detection.

3.2.2 End-to-End Event Lifecycle

Before collection begins, validated configuration, policies and detector definitions determine the logical event scope. The probe registry expands that scope into public producers and any internal support probes, while unrelated programs remain disabled. During monitoring, a selected hook produces a binary record that is transported, decoded against the event registry and admitted or rejected by the remaining user-space policy constraints. Admitted events are then routed independently to presentation and detector evaluation. A matched point condition or bounded sequence produces an alert that retains its source evidence and available MITRE ATT&CK metadata.

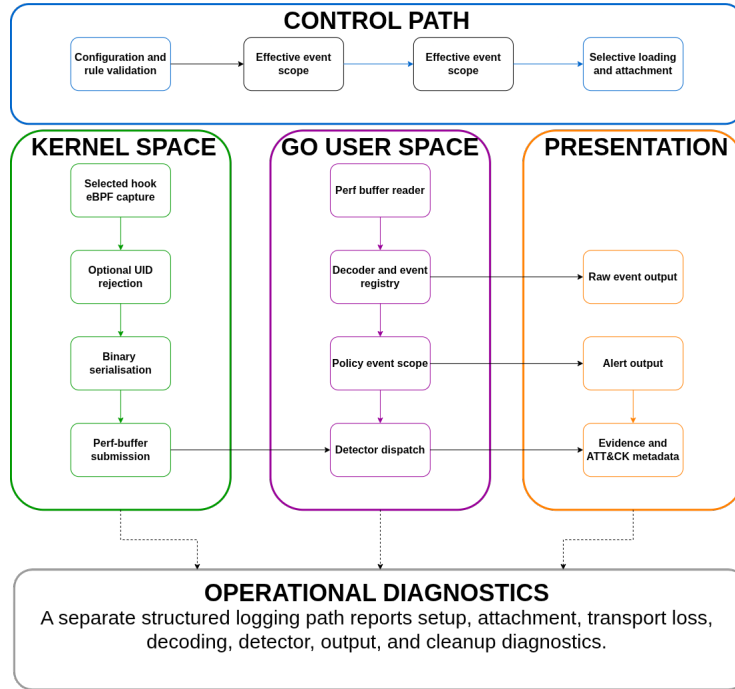


Figure 3.2: Responsibility boundary and end-to-end flow of the proposed monitoring system.

Figure 3.2 summarises the relationship between the control path and the monitoring data path. Operational diagnostics remain separate because they describe conditions affecting these paths rather than representing monitored events or detector alerts.

The diagram does not imply a total ordering of monitored events. Per-event transport uses per-CPU buffering, and collective detectors therefore interpret sequence order according to the order in which records reach the user-space engine. The corresponding transport-loss and ordering boundaries are discussed further in Section 3.4.2.

3.3 Kernel-Space Collection Design

Kernel instrumentation must expose evidence that is both meaningful for the logical event and obtainable on the target kernel. The design therefore does not prescribe one universal attachment mechanism. It combines available observation points while maintaining a stable event abstraction above them.

3.3.1 Hook Selection and Logical Events

Hook selection begins with the semantics required from the observation. The chosen point must expose the relevant stage of the operation, which may be the initial request, a resolved kernel object, a state transition, or a security mediation decision. An entry-side observation is sufficient when the attempted operation is itself meaningful. When the distinction between success and failure affects the interpretation of the event, a compatible return-side observation is also required.

The choice is further constrained by the observation points available on the concrete target and by their expected cost and robustness. An explicit tracepoint is preferable when it exposes the required semantics, whereas a dynamic probe may extend coverage where no suitable tracepoint exists, at the cost of stronger dependence on kernel symbols, function signatures, and calling conventions. Similarly, a dedicated observation point is preferred over a high-frequency generic hook when it provides equivalent evidence more selectively. These are design-time decisions derived from the properties of the kernel mechanisms discussed in Chapter 2, rather than choices made automatically while the system is running.

Table 3.3 distinguishes the architectural concepts used to describe instru-

mentation and event production.

Concept	Architectural meaning
Physical hook	A specific point in kernel execution, such as a tracepoint or kernel function, at which an eBPF program can be attached and invoked.
Producer probe	An eBPF program that directly captures an observation and submits a record for a public logical event. A logical event may be produced by one or more producer probes.
Support probe	An internal eBPF program that does not produce a public event of its own. Instead, it stores, derives, or supplies information required by a producer probe to construct another event correctly.
Logical event	The stable user-space event type exposed by the monitoring system, identified by a public name and decoding schema. It hides the number and type of physical hooks and probes used to construct the observation.

Table 3.3: Distinction between physical hooks, producer and support probes, and logical events.

The relationships between these concepts are not necessarily one-to-one. A logical event may require an entry probe to preserve request information and a return probe to complete the record with the operation outcome. Different operation variants may also converge on the same logical event. Conversely, selecting one public event may activate support probes that maintain required context but do not produce independently decodable records. Separating these attachment units from the public event abstraction prevents target-specific probe mechanics from becoming part of the user-facing event contract.

3.3.2 Early Event Selection and Kernel-Side Filtering

The public event abstraction introduced in the previous subsection also defines the unit of early selection. Rather than requiring the operator to identify individual eBPF programs, the architecture starts from the logical events that the monitoring scenario is expected to produce. For each selected event, the probe registry determines the producer probes responsible for its construction and any support probes on which those producers depend.

The resulting dependency set determines which registered programs enter the loading lifecycle. Programs that cannot contribute to the selected event domain are excluded before they are submitted to the kernel for verification and loading, and are consequently not attached to their hooks. This approach reduces the active observation surface while preserving the internal probes needed to construct each requested event. It also keeps the public selection interface independent from the particular arrangement of hooks and programs used on the target kernel.

Selective loading applies only to programs represented by the probe registry. It is not a general removal of every unused section contained in the compiled eBPF object, and its correctness depends on the registry describing producers and support dependencies completely. An omitted dependency could leave a selected event without information required for its construction, whereas an incorrectly exposed support probe could make an internal mechanism appear as an independently selectable event.

A further reduction can be applied after an attached program is invoked. The optional exact UID guard compares the effective UID of the current task with one configured value and rejects non-matching observations before complete context enrichment, record serialisation, and transport. The guard therefore avoids part of the kernel-side and transport work associated with observations known to be outside that narrow scope.

The guard is intentionally limited to a single equality condition and serves as a representative form of inexpensive kernel-side pre-filtering. Additional predicates over values such as process or local cgroup identity could be introduced where they are available reliably and provide a measurable reduction in unnecessary collection. More expressive conditions remain in user space, where they can be evaluated over the complete normalised event without increasing verifier-sensitive kernel logic.

3.4 Event Contract and Transport Design

Kernel-side producers and the user-space consumer require a common representation that remains consistent across heterogeneous event sources. The design combines a bounded binary transport contract with an explicit registry for semantic interpretation. These elements must evolve together: omitting repeated schema metadata from each record reduces transport redundancy, but also means that identifiers, encodings, and registry definitions

must remain aligned.

3.4.1 Stable Binary Event Contract

Each public record contains a common event context followed by indexed, typed arguments. The context carries the identity and provenance information shared across event families, including process, host, namespace, and execution context. Event-specific arguments describe the observed operation, while their indices associate the encoded values with the schema maintained by the user-space event registry.

The representation is intentionally not fully self-describing. The kernel producer emits an event identifier and the encoded argument values, whereas the event registry supplies the corresponding public name, argument types, and expected ordering. This avoids transporting the same semantic meta-data with every observation, but creates an explicit compatibility obligation between the kernel definitions, the Go protocol representation, the decoder, and the event registry.

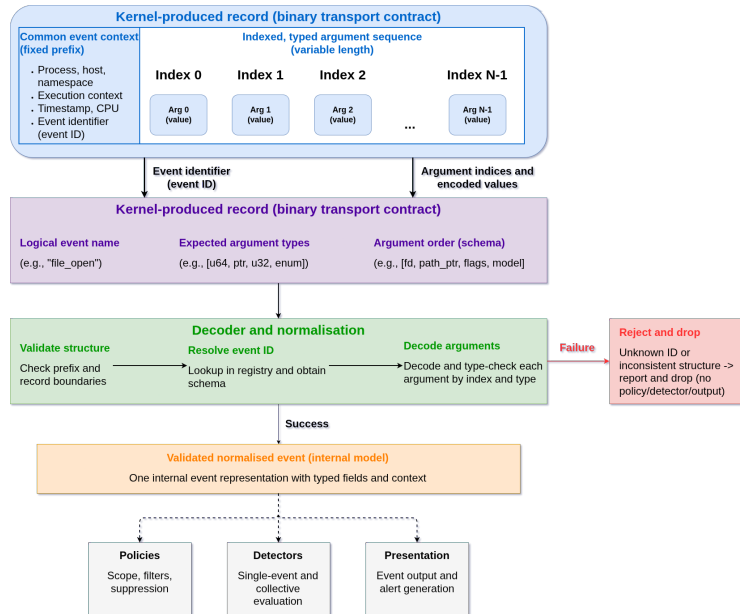


Figure 3.3: Conceptual relationship between the binary event record, the user-space event registry, and the resulting normalised event.

The contract remains valid only while producer and consumer agree on the context layout, event identifiers and indexed argument schemas. Unknown or structurally inconsistent records are discarded before policy, detector or output processing. The probe registry does not define this schema, but every public producer must refer to a logical event that the event registry can decode. Chapter 4 describes the producer and wire format; Chapter 5 describes the user-space registries and admission path that preserve this agreement.

3.4.2 Perf-Buffer Transport and Failure Boundaries

The operational event path uses perf-event transport, in which records are delivered through per-CPU perf ring buffers (`BPF_MAP_TYPE_PERF_EVENT_ARRAY`). This mechanism should not be confused with the BPF ring-buffer facility represented by `BPF_MAP_TYPE_RINGBUF`. Although both use circular buffering, the former maintains separate buffers for individual CPUs, whereas the latter provides a shared multi-producer ring buffer [54, 44].

Compatibility with the target kernel was the principal reason for selecting perf-event transport. The perf-event interface is available on the deployed Rocky Linux environment, whereas the BPF ring buffer belongs to a later upstream kernel generation and cannot be assumed to be present through a vendor backport. The transport used by every event was therefore based on the interface verified on the concrete target.

The BPF ring buffer offers advantages on compatible kernels, including shared capacity across CPUs and stronger ordering between records submitted to the same buffer. Perf-event transport instead uses per-CPU buffers, which may use memory less efficiently under uneven load and do not provide a total ordering across CPUs. These properties were accepted in favour of target compatibility and a simpler operational design.

Perf-event delivery remains asynchronous and finite. Records may be lost when a per-CPU buffer cannot be consumed quickly enough, and the resulting loss notification does not allow the missing observations to be reconstructed. A delivered record may instead fail during decoding if it violates the active binary contract. Transport loss and decoding failure are therefore separate conditions, neither of which implies that the corresponding kernel operation did not occur.

The per-CPU organisation also limits ordering guarantees. Collective de-

tectors evaluate sequences according to the order in which records reach the user-space engine, since the architecture does not reconstruct a total causal order across CPUs. The effects of transport pressure, loss, and concurrent delivery are assessed in Chapter 6.

3.5 User-Space Runtime Design

The Go runtime connects kernel collection with the analytical pipeline. Its design separates lifecycle management, record interpretation and presentation so that each resource and failure has a clear owner. Chapter 5 describes the corresponding implementation.

3.5.1 Loading, Attachment and Runtime Lifecycle

The runtime lifecycle separates validation and setup from active monitoring and cleanup. Figure 3.4 shows this progression and distinguishes failures that prevent a valid monitoring session from conditions that affect only one record.

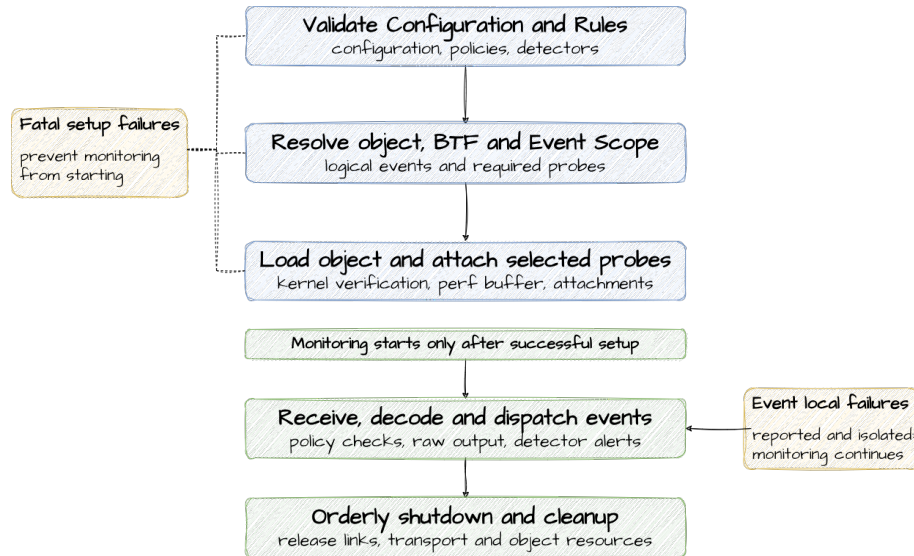


Figure 3.4: Simplified user-space runtime lifecycle.

Configuration and external definitions are validated before kernel resources are acquired. The accepted monitoring scope is then translated into the eBPF programs and detector instances needed for the session. Setup failures abort startup, whereas record-local failures are reported without terminating the event loop. Resources acquired by the runtime remain under its ownership and are released when monitoring ends, including after partial initialisation.

3.5.2 Decoding, Normalisation and Event Registration

The decoder is the validation boundary between transported bytes and the typed events used by the rest of user space. It interprets the common context and indexed arguments through the event registry, producing one normalised event representation for policies, detectors and output. Records that cannot be reconciled with a known schema are rejected at this boundary rather than being partially interpreted by downstream components.

The event and probe registries remain distinct. The event registry defines the public identifiers, names and argument schemas required for decoding, while the probe registry defines how logical events are produced from physical programs and support dependencies. This separation keeps wire interpretation independent from attachment planning, while requiring the two registries to remain aligned. Chapter 5 owns the concrete decoder behaviour and consistency checks.

3.5.3 Event Output, Alert Output and Operational Logging

The architecture exposes monitored events, detector alerts and operational diagnostics as three separate result classes. Event output presents admitted telemetry, alert output presents successful detector results and diagnostics describe the health of the monitoring infrastructure. Disabling ordinary event presentation must not disable detector evaluation, and diagnostic failures must not be mistaken for monitored behaviour or security alerts. Chapter 5 specifies the output formats, logging implementation and record-local failure handling.

3.6 Policy and Detection Architecture

The analytical subsystem separates configuration-time planning from event-time evaluation. During bootstrap, policy and detector documents are validated and translated into an execution plan. During monitoring, that plan admits selected events and evaluates them without changing the active kernel instrumentation.

3.6.1 Policy-Based Event Scope

Policies define the observation domain rather than the conditions that generate an alert. When their event scope is finite, it may reduce collection before the eBPF object is loaded; an open-ended scope requires the broader collection plan to be retained. Policies may also select the detector catalogue through descriptive metadata. Conditions that depend on decoded event data remain user-space admission rules. A rejected event reaches neither event output nor detectors, while the absence of policies leaves this boundary permissive.

This model produces one effective monitoring plan rather than an independent pipeline for each policy. It also preserves a clear distinction: policies decide which observations and detectors are relevant to a scenario, whereas detectors decide whether admitted evidence constitutes an alert. Chapter 5 describes how policy documents are compiled and evaluated.

3.6.2 Point and Collective Detectors

The terms *point* and *collective* describe how much event evidence a detector requires. They do not denote learned anomaly models in this system. A point detector evaluates one observation, whereas a collective detector evaluates whether several observations satisfy an explicitly configured local sequence. Both forms are deterministic and rule based; neither learns a baseline or assigns a statistical anomaly score.

Both detector classes consume the same normalised event representation used by policy matching. A point detector evaluates one event and can produce an alert immediately. A collective detector evaluates an ordered local sequence and therefore owns bounded partial-match state. Their difference lies in the amount of evidence and state required, not in separate processing pipelines.

Detector definitions declare the events they consume but do not attach probes. The required observations must be included in the monitoring plan before the detector engine is constructed. This one-way dependency prevents event-time detection from modifying kernel instrumentation.

3.6.3 Bounded Local Correlation

Collective detectors relate observations through explicit local identities. Process identity combines a host process identifier with its start time; parent relationships extend this scope only to an immediate process relation. File-resource identity uses a device and inode pair, while cgroup identity uses the node-local kernel identifier. Composite correlation requires all configured dimensions to agree. These keys do not represent persistent process graphs, arbitrary ancestry or Kubernetes workload identity.

Partial sequences are retained for a finite interval and under a finite per-detector capacity. Correlation dimensions that cannot be supported by the event model are rejected instead of being approximated from weaker evidence. The design therefore supports bounded local sequences without becoming a general-purpose complex-event processing system [16, 1]. Chapter 5 owns the exact state transitions, limits and pruning behaviour.

3.6.4 Alert Evidence and MITRE ATT&CK Metadata

An alert preserves the detector identity, severity and evidence that completed the match: one event for a point detector or an ordered event set for a collective detector. This evidence remains part of the analytical result rather than being reconstructed from presentation output.

MITRE ATT&CK tactics and techniques belong to detector metadata. Policies may use them when selecting the detector catalogue, and successful alerts carry the same classification for interpretation and downstream organisation [29]. The mapping does not replace detector predicates or establish correctness and coverage; Chapter 5 describes its validation and propagation through the implemented pipeline.

3.7 Design Decisions and Scope Boundaries

The architecture resolves three principal trade-offs. The first is between observation breadth and target reliability. Selective loading, explicit probe dependencies, and validation against the deployed enterprise kernel are preferred to assuming that every registered event can be observed portably. The resulting design supports the stated Rocky Linux target without claiming universal compatibility.

The second trade-off is between early reduction and analytical flexibility. Small, verifier-compatible filters may prevent irrelevant records from crossing the kernel boundary, but policy interpretation and detector semantics remain in Go so that they can evolve independently from eBPF compilation. The explicit event contract makes this division possible, while the selected perf-event transport reflects the capabilities validated for the reference environment.

The third trade-off is between richer security interpretation and predictable runtime state. Deterministic point and bounded collective detectors provide more focused results than raw telemetry alone without introducing persistent graphs, statistical learning, or unbounded correlation. The prototype is therefore an observation and detection system rather than an enforcement or general-purpose event-processing engine. Its claims are limited to the event coverage, local relationships, ATT&CK mappings, compatibility, and performance profiles evaluated in Chapter 6.

Container packaging supports later operational use but is a user-space and deployment concern described in Chapter 5. A wider platform may eventually aggregate alerts from several monitored hosts and add deployment context, but that external capability is neither implemented nor evaluated as part of this thesis. Within its stated boundary, the proposed system provides a coherent path from selected kernel observations to normalised telemetry and evidence-preserving local detections on the identified enterprise kernel.

Chapter 4

Kernel-Space Event Collection Implementation

This chapter describes how the kernel-space collection design developed in Chapter 3 is realised in the prototype. It covers the build artefacts, shared event-construction mechanisms, implemented hook families, binary record contract, perf-buffer transport, early UID admission and compatibility work required by the target Rocky Linux kernel. Chapter 5 follows the record into the Go runtime and describes loading, decoding, policy evaluation, detection and presentation. Experimental behaviour and performance measurements are reserved for Chapter 6.

4.1 Build Toolchain and Project Organisation

The build process consists of three ordered stages. First, the libbpf source in `3rdparty/libbpfgo/libbpf/src` is compiled locally into the static library `dist/libbpf/obj/libbpf.a`, together with the required headers and package metadata. Second, Clang compiles the eBPF source into `dist/project.\gls{bpf}.o` for the x86-64 target. Finally, the Go application is built through CGO using the local libbpf library and libbpfgo bindings, producing the executable `dist/project`. The Makefile preserves this order and ensures that all stages use the dependencies included in the repository rather than system-installed versions.

The eBPF object is also embedded in the Go executable. For this reason,

`dist/project.\gls{bpf}.o` must be generated before the Go build starts. At runtime, the application can load the embedded object without requiring a separate copy beside the executable. Developers may still provide an external object through the command-line configuration for testing or compatibility analysis. The executable nevertheless remains dependent on the host kernel, its BTF data, the required privileges and the system libraries used by CGO.

4.2 eBPF Program Implementation

The eBPF object contains programs attached to different kernel hook types. Most of these programs produce records in one of two ways. A direct producer creates and submits an event during a single hook invocation. A paired producer saves the request arguments when an operation begins and completes the event when a return hook provides its result.

Physical hooks and public events do not have a one-to-one relationship. Several eBPF programs may contribute to the same logical event and use the same event schema. Conversely, some attached programs only provide internal support and do not generate a public event. Section 5.2 explains how the loader translates these relationships into an attachment plan.

4.2.1 Shared Event-Construction Framework

Both construction forms use a per-CPU workspace to build events. Each CPU has its own reusable record, avoiding the need to place the complete event structure on the limited eBPF stack. Additional per-CPU maps provide temporary storage for path reconstruction and intermediate kernel objects, while task and process maps retain context that can be reused across hook invocations.

The common initialisation helper clears the argument buffer and records the basic identity and timing information. Each hook then appends its event-specific fields. Immediately before submission, the common submission helper updates the complete task context and sends the record through the kernel-to-user-space interface described in Section 4.3.

Listing 4.1 shows the main stages shared by these helpers. Details related to process-map initialisation and optional stack capture are omitted because they do not change the event-construction process.

```
1  if (!p->event)
2      p->event = bpf_map_lookup_elem(&event_data_map, &zero);
3  if (!p->event)
4      return 0;
5
6  p->config = bpf_map_lookup_elem(&config_map, &zero);
7  if (!p->config)
8      return 0;
9
10 reset_event_args_buf(p->event);
11 p->event->task = (struct task_struct *) bpf_get_current_task();
12
13 u64 id = bpf_get_current_pid_tgid();
14 p->event->context.task.host_tid = id;
15 p->event->context.task.host_pid = id >> 32;
16 p->event->context.eventid = event_id;
17
18 init_task_context(&p->event->context.task,
19                  p->event->task, p->config->options);
20
21 u32 size = sizeof(event_context_t) + sizeof(u8) +
22           p->event->args_buf.offset;
23 long ret = bpf_perf_event_output(
24     p->ctx, &events, BPF_F_CURRENT_CPU, p->event, size);
25
26 update_event_stats(p->event->context.eventid, ret);
27 return ret;
28
```

Listing 4.1: Shared event construction and submission path.

The per-CPU workspace is reused after each event submission. When the submission helper is called, the completed portion of the record is copied to the perf-event channel, so later modifications to the workspace do not affect the submitted event. If the program cannot obtain the required workspace, configuration or task context, it stops processing that invocation without producing a record.

The complete implementation also verifies the record size before submission and optionally updates delivery statistics. The listing therefore focuses on the point at which the event leaves kernel space. The precise size checks and error-handling rules are described in Section 4.3.

Paired producers build one event across two hook invocations. The entry program stores the request arguments in a shared map, while the return program retrieves them and adds the operation result. Each map entry is

identified by the logical event and the host thread that issued the request. This allows the return hook to recover the correct arguments without mixing different operations performed by the same thread.

```
1  statfunc u64 get_args_id(u32 event_id)
2  {
3      return ((u64) event_id << 32) |
4             (u32) bpf_get_current_pid_tgid();
5  }
6
7  statfunc int save_args(args_t *args, u32 event_id)
8  {
9      u64 key = get_args_id(event_id);
10     bpf_map_update_elem(&args_map, &key, args, BPF_ANY);
11     return 0;
12 }
13
14 statfunc int load_args(args_t *args, u32 event_id)
15 {
16     u64 key = get_args_id(event_id);
17     args_t *saved = bpf_map_lookup_elem(&args_map, &key);
18     if (!saved)
19         return -1;
20
21     /* Fixed argument-slot copies are omitted. */
22     return 0;
23 }
```

Listing 4.2: Saving and recovering arguments for paired producers.

Including the event identifier in the key prevents different paired operations from using the same map entry. The implementation copies the seven argument slots explicitly, keeping every memory access statically bounded for the eBPF verifier. If no entry is found, the return program cannot reconstruct the event and therefore produces no record.

After the saved arguments have been copied, the return program removes the map entry and appends the operation result. New entry data is stored with `BPF_ANY`, allowing it to replace stale state left by a missing return hook. The map is therefore temporary correlation storage, not a history of process activity.

The following subsections apply these two construction patterns to different hook families. The interpretation of each event depends on where the hook is placed within the kernel operation. A syscall-entry hook records the operation requested by a process, but not its outcome. A security hook observes

the operation while it is being checked by the kernel, without necessarily revealing the final decision. A committed-state hook runs after a new state has been installed, while a lifecycle tracepoint reports that a process transition has occurred. An exit or return hook completes the observation by recording the result available at that boundary, such as a returned value or error code.

4.2.2 Process Lifecycle and Executable Transitions

Process monitoring must distinguish between a creation request and the lifecycle transition produced by the kernel. Syscall events expose the operation selected by the process and the result returned at the syscall boundary. Scheduler tracepoints instead observe tasks that have already been created, renamed or terminated. The implementation combines these sources because neither view is sufficient on its own.

4.2.2.1 Process Creation and Lifecycle

The `fork` and `vfork` events provide the simplest syscall-level view of process creation. Their schemas contain only the return value, so their entry probes do not retain any arguments. A positive result identifies the child returned to the caller, while a negative result records the reported error.

The `clone` event captures a more configurable creation request. Its entry probe retains the clone flags, stack address, parent and child thread-ID pointers, and thread-local-storage value. These fields describe which resources the new task should share with its parent and where its execution context should be initialised. The exit probe combines them with the returned child identifier or error. The resulting event therefore preserves both the requested creation model and the syscall outcome.

Syscall results alone do not provide the complete parent-child state observed by the scheduler. The `sched_process_fork` tracepoint runs after the child task has been created and receives both task structures. It records the host PID and TID of the parent and child, together with the child's start time. Listing 4.3 shows the extraction of this effect-level evidence.

```
1 struct task_struct *parent =  
2     (struct task_struct *) ctx->args[0];  
3 struct task_struct *child =  
4     (struct task_struct *) ctx->args[1];
```

```
5
6 u32 parent_pid = get_task_host_tgid(parent);
7 u32 parent_tid = get_task_host_pid(parent);
8 u32 child_pid = get_task_host_tgid(child);
9 u32 child_tid = get_task_host_pid(child);
10 u64 child_start = get_task_start_time(child);
11
12 save_to_submit_buf(&p.event->args_buf,
13                  &parent_pid, sizeof(u32), 0);
14 save_to_submit_buf(&p.event->args_buf,
15                  &parent_tid, sizeof(u32), 1);
16 save_to_submit_buf(&p.event->args_buf,
17                  &child_pid, sizeof(u32), 2);
18 save_to_submit_buf(&p.event->args_buf,
19                  &child_tid, sizeof(u32), 3);
20 save_to_submit_buf(&p.event->args_buf,
21                  &child_start, sizeof(u64), 4);
```

Listing 4.3: Parent-child evidence collected by `sched_process_fork` (abridged).

The start time complements the numeric child identifiers. PIDs and TIDs may be reused after a task terminates, while the start time allows user space to distinguish separate task lifetimes associated with the same identifier. This combination also provides a stronger process identity for local detector correlation.

Two direct tracepoints cover later lifecycle changes. `task_rename` records the previous and new command names when the task name changes. This operation is kept separate from executable replacement because a command-name change does not imply that a new program image was installed. The `sched_process_exit` event records the task's exit code and whether the complete thread group is terminating, distinguishing a single thread exit from the end of the process.

The implementation emits these lifecycle relations as bounded events rather than maintaining a persistent process graph in kernel space. Syscall events describe requests and returned outcomes, whereas scheduler tracepoints provide evidence of task states visible after the corresponding kernel transition. Longer-lived reconstruction remains a user-space responsibility.

4.2.2.2 Executable Transitions

Executable-transition monitoring must preserve both the program requested by a process and the outcome of the replacement operation. The pathname and arguments available at syscall entry describe the process's intention, but they cannot establish whether the kernel accepted the executable or installed the new image. The implementation therefore observes the request, binary security processing and final outcome at separate kernel boundaries.

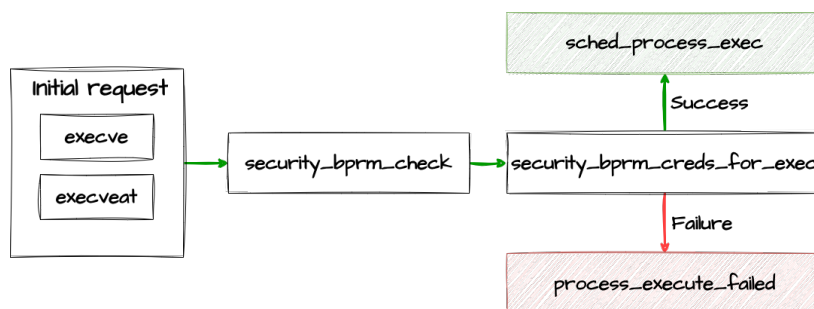


Figure 4.1: Events used to observe execution requests, kernel security processing and executable-transition outcomes.

Figure 4.1 represents monitoring coverage rather than a guaranteed one-to-one event sequence. Binary-handler and interpreter processing may cause security-stage observations to occur more than once. Furthermore, the security hooks identify intermediate kernel processing but do not expose the final security decision.

The initial request is captured through `execve` and `execveat`. Both events preserve the requested pathname and up to 38 argument strings while the original user-space memory is still available. The `execveat` event additionally records the directory file descriptor and execution flags, which are required to interpret relative paths and descriptor-based execution. These events are emitted at syscall entry and therefore do not contain an execution result.

The `security_bprm_check` hook observes the executable after the kernel has constructed its `linux_binprm` representation. At this stage, the implementation can record the resolved pathname, device and inode, original filename, argument count and environment count. These fields associate the request with a resolved kernel object rather than relying only on the

caller-supplied path.

The `security_bprm_creds_for_exec` hook provides a related view of credential preparation. It captures the same binary identity fields and adds the file change time. This observation does not prove that the prepared credentials became active; committed credential transitions are monitored separately by the mechanism described in Section 4.2.3.1.

The final outcome is represented by two mutually exclusive logical events. A `sched_process_exec` event confirms that the process installed a new executable image and records its resolved filename. An unsuccessful `execve` or `execveat` instead produces `process_execute_failed`, which retains the operation variant, directory file descriptor, requested pathname, flags and negative return value.

The failure event uses paired entry and exit programs because the pathname and execution options are available before the syscall runs, while the error is available only when it returns. Listing 4.4 shows the return-side selection that suppresses successful executions.

```
1  args_t args = {};
2  if (load_args(&args, PROCESS_EXECUTE_FAILED) != 0)
3      return 0;
4
5  del_args(PROCESS_EXECUTE_FAILED);
6
7  if (ret >= 0)
8      return 0;
9
10 program_data_t p = {};
11 if (!init_program_data(&p, ctx, PROCESS_EXECUTE_FAILED))
12     return 0;
13
14 int operation = (int) args.args[0];
15 int dirfd = (int) args.args[1];
16 const char *pathname = (const char *) args.args[2];
17 int flags = (int) args.args[3];
18
19 save_to_submit_buf(&p.event->args_buf,
20                  &operation, sizeof(operation), 0);
21 save_to_submit_buf(&p.event->args_buf,
22                  &dirfd, sizeof(dirfd), 1);
23 if (pathname)
24     save_str_to_buf(&p.event->args_buf, pathname, 2);
25 save_to_submit_buf(&p.event->args_buf,
26                  &flags, sizeof(flags), 3);
27 save_to_submit_buf(&p.event->args_buf,
```

```
28         &ret, sizeof(ret), 4);  
29  
30 return events_perf_submit(&p);
```

Listing 4.4: Return-side emission of failed executable transitions (abridged).

The saved entry state is removed before the return value is evaluated. A successful result therefore clears the temporary map entry without generating a failure record, while a negative result is serialised with the original request. Separate entry and exit programs handle `execve` and `execveat`, but all four physical programs converge on the same public event schema.

This design keeps caller intent, resolved binary identity and observed outcome as distinct forms of evidence. It avoids interpreting an execution attempt as a successful transition and allows user space to present negative return values as readable execution errors.

4.2.3 Identity, Credentials, Process Control and Limits

4.2.3.1 Identity and Credentials

A privilege transition cannot be reconstructed reliably from a set-ID syscall alone. The syscall identifies the change requested by the process and reports its result, but credentials may also change during executable loading or through internal kernel paths. Conversely, a security hook exposes a proposed transition without proving that the new state became active. The implementation therefore separates requested identity changes, security mediation, committed credentials and subsequent capability checks.

Linux associates each process with several user and group identifiers. The real ID represents the identity under which the process was started, while the effective ID is used for most access-control decisions. The saved ID allows a process to retain an effective identity that may later be restored, supporting temporary privilege reduction and recovery. Linux also maintains filesystem IDs, which are used specifically during filesystem permission checks. The same distinction applies to user and group credentials [18].

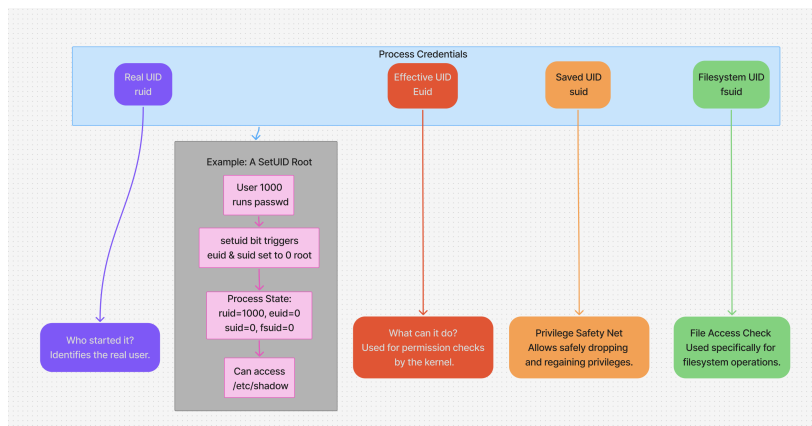


Figure 4.2: Real, effective, saved and filesystem UIDs for a process [18].

Monitoring all these values is security-relevant because a process may change its effective or filesystem privileges without changing its real identity. Saved identities may also allow privileges to be regained after an apparent drop. Observing only the real ID would therefore hide important transitions in the process's effective authority [18].

Explicit identity requests are captured through the `setuid`, `setgid`, `setreuid`, `setregid`, `setresuid`, `setresgid`, `setfsuid` and `setfsgid` events. Their entry programs retain one, two or three identifiers according to the syscall interface. This preserves the distinction between single-ID changes, separate real and effective identities, complete real/effective/saved transitions and filesystem-specific identities. The exit programs append the value returned to the caller.

The return value cannot be interpreted uniformly across the family. Most set-ID operations use a conventional status value, while `setfsuid` and `setfsgid` return the previous filesystem identity. Preserving the originating logical event is therefore necessary for user space to interpret the result correctly.

The `security_task_fix_setuid` hook adds the mediation-stage view. It records the old and proposed real, effective, saved and filesystem UIDs, together with a flag that distinguishes the set-ID operation category. The eBPF program compares the two credential proposals and suppresses the event when none of the monitored UID fields changes. Since the hook is observed before installation and its result is not captured, the event describes

the transition examined by the security subsystem rather than a confirmed change.

No public `security_task_fix_setgid` event is currently exposed. Group-ID requests remain visible through the set-group syscall events, while changes that become active are included in the broader `commit_creds` event.

The `commit_creds` hook observes the point at which replacement credentials become active for the current task. It reads both the currently installed credentials and the replacement supplied by the kernel, converting them into fixed-width snapshots. Each snapshot contains real, effective, saved and filesystem user and group identifiers, the user-namespace identifier, secure bits, and the inheritable, permitted, effective, bounding and ambient capability sets.

Credential installation is a frequent kernel operation, and not every invocation changes a field relevant to the monitor. The implementation therefore compares the two snapshots before submission. Listing 4.5 shows this change-only emission strategy.

```
1 slim_cred_t old_slim = {};
2 slim_cred_t new_slim = {};
3
4 /* Identifiers, namespace, secure bits and capability
5  * sets are copied into the two snapshots. */
6
7 if (old_slim.uid == new_slim.uid &&
8     old_slim.gid == new_slim.gid &&
9     old_slim.euid == new_slim.euid &&
10    old_slim.egid == new_slim.egid &&
11    /* Remaining monitored fields are compared here. */
12    old_slim.cap_ambient == new_slim.cap_ambient)
13     return 0;
14
15 save_to_submit_buf(&p.event->args_buf,
16                  &old_slim, sizeof(slim_cred_t), 0);
17 save_to_submit_buf(&p.event->args_buf,
18                  &new_slim, sizeof(slim_cred_t), 1);
19
20 return events_perf_submit(&p);
```

Listing 4.5: Kernel-side suppression of unchanged credential snapshots (abridged).

The comparison occurs before the event enters the perf-event channel. Unchanged credential installations therefore consume neither transport band-

width nor user-space decoding work. When a relevant difference is found, both snapshots are preserved, allowing the transition to be analysed without reconstructing the previous state from an earlier event.

The final observation point is `cap_capable`. This event records the capability number presented during a kernel capability check, exposing the use of privilege-sensitive operations even when no credential replacement occurs. A capability check is not equivalent to a privilege change or a successful operation: the current schema does not retain the checked credentials, user namespace, option flags or final decision.

Together, these events distinguish four forms of evidence: an explicit identity request, a proposed transition under security mediation, the credential state installed by the kernel and a later capability check. This prevents the monitoring pipeline from treating all privilege-related activity as the same kind of event.

4.2.3.2 Process Control and Limits

Process-control operations can alter another task, modify the behaviour of the current process or change the limits within which it executes. These operations are security-relevant because they may support inspection, interference, persistence or resource manipulation. The implementation observes them at different boundaries according to where their most useful evidence becomes available.

Cross-process control is represented by `ptrace` and `security_task_kill`. The `ptrace` entry program retains the request code, target PID and the address and data arguments supplied by the caller. These pointer values are deliberately treated as opaque: the eBPF program does not copy the memory to which they refer. At syscall exit, the stored request is combined with the return value, allowing user space to distinguish an accepted request from an error.

Signal delivery is observed through `security_task_kill`. The common event context already identifies the source task, while the event-specific payload describes the destination through its host PID and TID, UID and command name. The signal number completes the source-operation-target relation. Signal zero is retained because it can be used to test whether a target exists and whether the caller has permission to signal it, even though no signal is actually delivered.

Listing 4.6 shows how the destination identity is extracted from the target task rather than inferred from a caller-supplied numeric PID.

```
1  u32 target_pid = get_task_host_tgid(target);
2  u32 target_tid = get_task_host_pid(target);
3  u32 target_uid = 0;
4  char target_comm[TASK_COMM_LEN] = {};
5
6  const struct cred *target_cred =
7      get_task_real_cred(target);
8  if (target_cred)
9      target_uid = BPF_CORE_READ(target_cred, uid.val);
10
11  if (bpf_probe_read_kernel_str(
12      target_comm, sizeof(target_comm),
13      target->comm) <= 0)
14      return 0;
15
16  save_to_submit_buf(&p.event->args_buf,
17                  &target_pid, sizeof(u32), 0);
18  save_to_submit_buf(&p.event->args_buf,
19                  &target_tid, sizeof(u32), 1);
20  save_to_submit_buf(&p.event->args_buf,
21                  &target_uid, sizeof(u32), 2);
22  save_str_to_buf(&p.event->args_buf, target_comm, 3);
23  save_to_submit_buf(&p.event->args_buf,
24                  &sig, sizeof(int), 4);
```

Listing 4.6: Target evidence collected by `security_task_kill` (abridged).

Reading the destination directly from its kernel task structure avoids relying only on an identifier supplied by user space. However, this hook runs during security mediation and does not expose the final decision or confirm that the signal was delivered. It records an attempted relation rather than a completed state change.

Changes to the current task’s behaviour are monitored through `security_task_prctl` and `do_sigaction`. The former records the requested `prctl` option and its four machine-word arguments. The eBPF program intentionally leaves option-specific interpretation to user space, avoiding a large command-dependent parser in kernel space.

The `do_sigaction` event focuses specifically on signal-handler state. It records the signal number, whether a new action was supplied and whether the caller requested the previous action. For both the proposed and currently installed actions, it captures the flags, first signal-mask word and handler

category or address. This makes changes such as installing, replacing or ignoring a handler observable. Since the hook is attached at function entry, the event does not confirm that the proposed action was installed.

Resource-limit monitoring combines a mediation view with syscall-level outcomes. The `security_task_setrlimit` hook receives the resolved target task and proposed kernel `rlimit` structure. It records the target host PID, resource identifier, soft limit and hard limit while the operation is being checked. It does not include the final decision.

The paired `prlimit64` event instead preserves the target PID, resource identifier, addresses of the new and old user-space limit structures, and the syscall return value. The pointed-to structures are not dereferenced by this event. These two observations are therefore complementary: the security hook exposes the proposed kernel values, while the syscall event preserves the request interface and its reported outcome.

Finally, `security_settime64` observes attempts to modify the system clock. It records the proposed seconds and nanoseconds together with the time-zone offset and daylight-saving field. Changing wall-clock time is privilege-sensitive and can affect timestamp-based analysis, for example an attacker can leverage the time modification to obscure their activities or perform some time based TOTP authentication, but this mediation-stage event does not by itself establish that the new time was installed.

This design keeps cross-process relations, task configuration and resource changes as separate forms of evidence. Syscall return events provide explicit outcomes where available, while security hooks expose resolved kernel objects and proposed values without being treated as confirmation of success.

4.2.4 File and Filesystem Security

Filesystem monitoring must distinguish the name supplied by a process from the object eventually reached by the kernel. A pathname may be relative, interpreted from a directory file descriptor or contain symbolic links. The Virtual File System resolves these elements before operating on the corresponding filesystem object [53].

The implementation therefore combines two complementary perspectives. Syscall probes preserve the original request and its returned result, while security hooks observe objects that have already undergone kernel pathname resolution. Where available, the resolved device and inode numbers provide

a local object identity, since an inode number is unique only within its filesystem [27].

The **open** event provides the syscall-level perspective. Entry probes for **open** and **openat** retain the operation variant, directory file descriptor, requested pathname, flags and creation mode. Exit probes add either the returned file descriptor or a negative error. The two syscall variants therefore share one logical schema without losing the differences between their calling conventions.

The same design is used for filesystem metadata changes. The **chmod** event monitors requests to modify permission bits, which determine whether an object may be read, written or executed. It combines **chmod**, **fchmod** and **fchmodat**, recording the relevant pathname, descriptor or directory-relative context, the requested mode and the syscall result.

The **chown** event monitors requests to replace the user or group ownership of an object. It combines **chown**, **fchown**, **fchownat** and **lchown**, retaining the requested owner and group identifiers, pathname or descriptor context, flags and returned result. The operation identifier remains important because **lchown** changes the ownership of a symbolic link itself rather than following the link to its target.

This consolidation separates the public event model from physical attachment. Six entry and exit programs produce the single **chmod** event, while eight produce **chown**. A compact operation field identifies the originating syscall and avoids duplicating otherwise similar event schemas.

The syscall view is complemented by **security_file_open**. This hook is invoked while the kernel is opening a file, after the Virtual File System has resolved the pathname and constructed a **struct file**. Linux Security Modules may use this stage to preserve open-time security state and re-evaluate conditions previously checked through **inode_permission** [56]. The hook therefore exposes the resolved object under security inspection, but does not report the final LSM decision or syscall result.

From the supplied **struct file**, the program obtains the resolved pathname, file flags, device number, inode number and change time. When the originating syscall is recognised, it also attempts to recover the pathname argument from the current task register state. Listing 4.7 shows the essential separation between these two forms of evidence.

```
1 void *resolved = get_path_str(&file->f_path);
2 dev_t device = get_dev_from_file(file);
3 unsigned long inode = get_inode_nr_from_file(file);
4
5 void *requested = "";
6 struct pt_regs *regs = get_current_task_pt_regs();
7
8 if (p.event->context.syscall == SYSCALL_OPEN)
9     requested = get_syscall_arg1(p.event->task, regs, false);
10 else if (p.event->context.syscall == SYSCALL_OPENAT)
11     requested = get_syscall_arg2(p.event->task, regs, false);
12
13 save_str_to_buf(&p.event->args_buf, resolved, 0);
14 save_to_submit_buf(&p.event->args_buf, &device, sizeof(device), 2);
15 save_to_submit_buf(&p.event->args_buf, &inode, sizeof(inode), 3);
16 save_str_to_buf(&p.event->args_buf, requested, 5);
```

Listing 4.7: Requested and resolved evidence collected by `security_file_open`.

The requested and resolved paths answer different questions. The first describes how the process referred to the file; the second describes the path reconstructed from the kernel object. They may differ because of symbolic links, relative traversal or mount topology. Requested-path recovery is best-effort and may produce an empty value when the syscall or register state cannot be identified. The resolved object remains available directly through the `struct file`.

Path reconstruction is itself constrained by the eBPF execution model. The implementation walks kernel dentries backwards and assembles the path in per-CPU scratch storage rather than on the restricted eBPF stack. The traversal is bounded to 20 components and a 4096-byte working region, allowing the verifier to establish finite execution and memory bounds. Deep or disconnected paths may consequently produce a partial or explicitly disconnected representation. A resolved pathname must therefore be understood as bounded kernel evidence rather than an unconditional canonical path.

Device and inode values provide a more stable local correlation key than a pathname alone. Detectors can associate observations generated by different processes when they refer to the same `device:inode` pair, even if the path strings differ. The pathname remains useful as a detector condition, but does not define resource identity. This correlation is deliberately local and time-bounded because inode values can eventually be reused and layered filesystems may expose different identities for apparently equivalent paths.

Further file hooks observe access checks and control requests on resolved objects. `security_file_permission` records the pathname, device-inode identity, change time and permission mask presented during a read, write or execution check. It shows that the check occurred, but does not establish whether access was ultimately granted. This is also a high-volume observation point, making selective event activation particularly important.

The `security_file_ioctl` event monitors file and device-specific control requests. It records the resolved target identity, ioctl command and argument value. The argument is intentionally retained as an opaque value: its interpretation depends on both the command and the target device, and decoding every possible structure inside eBPF would increase complexity and verifier cost.

Directory-entry hooks preserve objects at the point where their namespace representation may change. `security_inode_unlink` records the path, device, inode and change time of an entry proposed for removal. Capturing this identity before the unlink operation is valuable because the pathname may no longer resolve afterwards.

The `security_inode_rename` event records the old and new paths, rename flags and the identity of the source object. Its implementation also reveals a consequence of bounded per-CPU storage. Both paths are reconstructed through the same scratch buffer, so the source path must be serialised into the event before reconstruction of the destination begins. Otherwise, the second traversal would overwrite the first result.

Creation-related hooks retain evidence specific to the new directory entry. `security_inode_symlink` records the pathname of the symbolic link and the caller-supplied target string, which may be relative or may not yet identify an existing object. `security_inode_mknod` records the pathname, requested mode and device number of a new special node. The mode identifies the object type and permission bits, while the device number is meaningful for character and block devices. Both events observe security mediation and do not contain a final syscall result.

Mount hooks extend monitoring from individual objects to filesystem topology. `security_sb_mount` records the source name, resolved target path, filesystem type and mount flags. The filesystem-specific data argument is not copied because its format varies between filesystems and may contain sensitive configuration. `security_sb_umount` records the resolved mount-point, filesystem type and unmount flags. As entry observations of security

hooks, these events show that the requested topology change reached mediation, not that it completed successfully.

The resulting implementation preserves three distinct forms of filesystem evidence: the operation requested by a process, the kernel object involved in security mediation and, for paired syscall probes, the result returned to the caller. Keeping these stages separate prevents a pathname request or an LSM observation from being presented as proof that the corresponding filesystem operation succeeded.

4.2.5 Memory, Namespace and Cgroup Transitions

Memory, namespaces and cgroups control different aspects of process execution. Memory mappings determine which regions a process may access and how they may be used. Namespaces determine which kernel resources a process can observe, while cgroups define its position within a resource-control and accounting hierarchy. Monitoring these mechanisms is security-relevant because they are involved in code loading, process-memory access and changes to isolation boundaries.

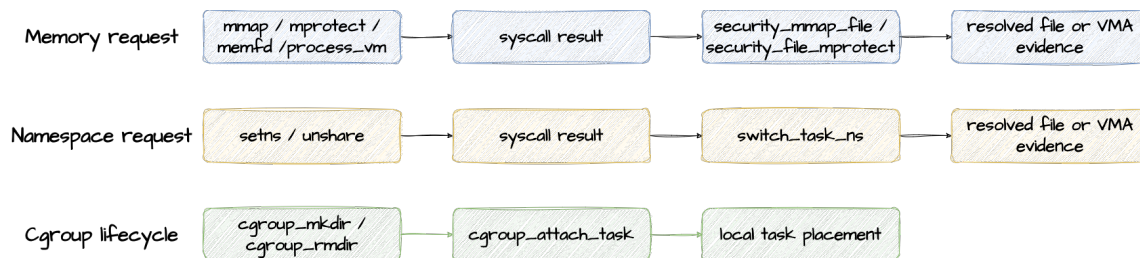


Figure 4.3: Observation points used for memory operations, namespace transitions and local cgroup changes.

Figure 4.3 represents complementary observation paths rather than a mandatory sequence. A memory syscall may not reach the corresponding security hook, and a successful namespace-related syscall does not necessarily modify every namespace represented by the committed-state event.

The `memfd_create` event monitors the creation of an anonymous, RAM-backed file. Such an object initially has no conventional filesystem pathname, but it may later be mapped into memory or used as an executable

image. The event records the requested name and creation flags, followed by the returned file descriptor or a negative error. This makes successful anonymous-file creation distinguishable from a failed attempt without requiring the monitor to inspect its future contents.

The `mmap` event records requests to create or modify mappings in a process address space. Its entry probe retains the requested address, length, protection mask, mapping flags, file descriptor and offset. The exit probe adds either the address selected by the kernel or a negative error. These fields distinguish anonymous mappings from file-backed mappings and identify requests that combine writable and executable permissions.

The `mprotect` event monitors requests to change the permissions of an existing memory range. It records the starting address, length, requested protection mask and returned status. The related `pkey_mprotect` event additionally records the protection key assigned to the range. Protection-key support is dependent on the target architecture and kernel configuration, so the returned value remains necessary for distinguishing an applied request from an unsupported or rejected operation.

The syscall events describe the process request, but they do not identify the resolved kernel objects examined during security mediation.

The `security_mmap_file` hook complements `mmap` for file-backed mappings. It records the resolved pathname, file flags, device, inode, change time, requested protections and mapping flags. Anonymous mappings are omitted because no backing `struct file` is available; they remain visible through the syscall event.

The `security_file_mprotect` hook observes a protection change through the virtual memory area supplied by the kernel. It preserves three distinct protection values: the permissions requested by user space, the permissions adjusted by the kernel for security evaluation, and the access bits currently installed on the region. It also records the region start and length. When the mapping is file-backed, the event adds its resolved pathname and device-inode identity; anonymous regions retain the protection and address evidence without file metadata.

This distinction makes the event more informative than the syscall request alone. It can show, for example, that a previously writable region is being evaluated for executable access and whether that region is associated with a resolved file. Since the probe observes entry to the security hook, however, it does not contain the hook decision or the final syscall result.

The implementation reads the supplied virtual memory area directly. It does not perform an independent search through the process memory-management tree. This keeps the collection path bounded and avoids duplicating work already performed by the kernel before invoking the security hook.

Cross-process memory access is monitored through `process_vm_readv` and `process_vm_writev`. These events record the target PID, addresses and element counts of the local and remote I/O-vector arrays, operation flags and the number of bytes transferred or the returned error. The vector elements and transferred memory contents are not copied. This retains evidence that one process accessed another process's address space while limiting overhead and avoiding collection of potentially sensitive payload data.

Namespace events monitor changes to the resource views associated with a process. The `setns` syscall requests that the caller join an existing namespace referenced by a file descriptor. Its event records the descriptor, the requested namespace type and the returned result. A type value of zero allows the kernel to infer the namespace type from the descriptor.

The `unshare` syscall instead requests that selected execution-context resources cease to be shared with other processes. Its event records the complete `CLONE_*` flag mask and the syscall result. Together, `setns` and `unshare` distinguish entry into an existing namespace from the creation or separation of an execution context.

A successful syscall result confirms that the request was accepted, but does not directly expose the identifiers installed in the task. The `switch_task_ns` event supplies this state-level perspective by observing `switch_task_namespaces`, where the kernel replaces the task's namespace proxy.

The hook compares the old and proposed namespace sets and emits an event only when at least one monitored identifier changes. The record contains the task PID and only the new identifiers for the changed mount, PID-for-children, UTS, IPC, network or cgroup namespaces. Unchanged fields are omitted from the indexed argument stream, keeping the record sparse and suppressing no-op transitions. User namespaces are not part of this namespace-proxy comparison; credential-related user-namespace changes are observed separately through the credential hooks.

Cgroup monitoring addresses task placement rather than namespace replacement. A cgroup namespace changes how a hierarchy is presented to a process, whereas cgroup attachment changes the hierarchy node to which the task belongs. The implementation preserves this distinction.

The `cgroup_attach_task` raw tracepoint reports movement of a task to a destination cgroup. It records the destination cgroup and hierarchy identifiers, the supplied cgroup path, target command name, host PID and TID, and whether the operation concerns the complete thread group. This allows the monitor to observe containment changes even when the task identity itself remains unchanged.

The `cgroup_mkdir` and `cgroup_rmdir` events describe the lifecycle of the local cgroup hierarchy. Both retain the cgroup identifier, bounded path and hierarchy identifier supplied at creation or removal. They do not reconstruct a persistent hierarchy in user space; each event represents a local lifecycle observation.

Cgroup identifier extraction also contains a compatibility decision specific to the supported kernel range. The layout of the underlying `kernfs_node` identifier differs between older kernels, newer kernels and the hidden KABI representation used by the target RHEL-derived kernel. CO-RE field-existence checks select the available layout at load time, allowing the same helper to obtain the identifier without hard-coding a single structure version.

Namespace inode numbers and cgroup identifiers remain local kernel identities. They are useful for comparing events and for detector correlation on one node, but they are not interpreted as pod, container or cluster-wide workload identifiers. The implementation therefore provides evidence of local memory and isolation transitions without claiming to reconstruct a persistent namespace graph, cgroup hierarchy or Kubernetes execution model.

4.2.6 Kernel Integrity and Kernel-Facing Activity

Kernel modules, eBPF objects and dynamic instrumentation can modify or inspect behaviour inside the kernel. These mechanisms are legitimate parts of Linux, but they also expose high-impact interfaces whose use is relevant to runtime security monitoring. The implemented hooks preserve the operation, object identity and result available at each observation point. They provide evidence for subsequent detectors and do not classify an operation as malicious by themselves.

Kernel-module monitoring separates lifecycle notifications from module initialisation results. The `module_load` raw tracepoint records a module as it enters the kernel module lifecycle, while `module_free` reports its removal.

Both events preserve the module name and, when available, its version and source-version strings. They do not copy module code, sections or other internal contents.

These lifecycle events do not indicate whether the module's initialisation function completed successfully. The `do_init_module` event therefore uses a kprobe and kretprobe pair. At function entry, the module object is retained in per-thread temporary state. At return, the implementation retrieves that object, extracts the module identity and appends the function result. A zero value identifies successful initialisation, while a negative value preserves the reported kernel error.

This distinction prevents a module observed during loading from being automatically treated as operational. The lifecycle tracepoints identify additions and removals, whereas `do_init_module` provides the outcome of the initialisation stage. These events can later be correlated without assigning a security meaning to ordinary module-management activity.

The eBPF hooks observe a different kernel-facing interface. They are kprobes attached to Linux Security Module functions invoked while eBPF commands and objects undergo security checks. They are not themselves BPF LSM enforcement programs and do not modify the kernel's decision.

The `security_bpf` hook monitors operations entering the security path of the BPF syscall. Its purpose is to expose requests such as map creation, program loading and program attachment. The event records the BPF command and attribute size, but does not copy `union BPF_attr`, whose active fields depend on the command.

The `security_bpf_map` hook observes security checks involving an existing eBPF map. Since maps retain state and exchange data between kernel programs and user space, identifying access to them provides useful object-level evidence. The event records the map identifier and bounded name, without collecting its contents.

The `security_bpf_prog` hook observes security operations involving an existing eBPF program. It records the program identifier, name and type, allowing programs intended for kprobes, tracepoints, cgroups and other execution contexts to be distinguished.

Together, these hooks identify the requested BPF operation and the map or program involved in security mediation.

Dynamic instrumentation and kernel-facing interfaces expose activity that

is not always visible through ordinary process syscalls. The following events monitor selected interfaces used for tracing, symbol resolution, procfs creation, kernel-initiated execution and kernel file consumption.

The `register_kprobe` event monitors attempts to install dynamic kprobes. It records the target symbol, pre- and post-handler addresses, and registration result. This distinguishes successful installations from rejected requests, although the symbol may be absent when registration uses a kernel address directly.

The `kallsyms_lookup_name` event monitors kernel symbol resolution. It combines the requested symbol name with the returned address, where zero indicates that no address was found. It covers this specific lookup interface rather than every possible source of kernel addresses.

The `proc_create` event observes requests to create procfs interfaces. It records the entry name and proc-operations address, making kernel-created user-space control or information endpoints visible. It does not include the parent directory, complete path or creation result.

The `call_usermodehelper` event monitors requests by the kernel to start a user-space program. It records the executable path, bounded argument and environment arrays, and wait mode. The arrays contain at most 38 elements and remain subject to the event-size limit. The event represents the request, not successful execution or completion of the helper.

The `security_kernel_read_file` event observes the security stage before the kernel consumes a file. It records the resolved pathname, device, inode, change time and read purpose, distinguishing uses such as modules, firmware, policy data and kexec images.

The later `security_kernel_post_read_file` event records the same file identity and purpose together with the supplied byte size. This size is not a return value. The two stages are emitted independently: the implementation does not correlate them automatically, copy file contents or compute a digest.

Together, these hooks expose selected instrumentation, procfs, kernel-initiated execution and kernel-file activity. They do not inspect handler code, reconstruct every kernel relationship, prove integrity or enforce a security decision. Chapter 5 explains how the corresponding logical events activate their physical producers and support dependencies.

4.3 Binary Event Contract and Kernel Transport

Every event crosses the kernel boundary in the same binary format. The record begins with a 136-byte context containing common process, host and event metadata. It is followed by a one-byte argument count and by the arguments captured for that specific event. The Go decoder mirrors the order and width of the C context fields. Only the populated part of the argument buffer is submitted, so an event does not carry the complete 32,000-byte workspace used while it is being constructed.

Arguments are identified by their position in the event schema. Each encoded argument begins with a one-byte index, followed by its payload. Fixed-width values are written directly, whereas variable-length values include a four-byte length before their content. Strings and argument arrays use the same principle with bounded internal records. The wire format does not need a separate type tag because the index tells the Go event registry which name and decoder to use. It also permits an unavailable argument to be omitted without renumbering the arguments that follow it.

Before writing an argument, the eBPF helper must prove to the verifier that the current offset and the complete payload remain inside the buffer. Listing 4.8 shows the essential checks.

```
1  if (size == 0 || size > MAX_ELEMENT_SIZE || buf->offset >=
    ↪ ARGS_BUF_SIZE)
2      return 0;
3  u32 offset = buf->offset;
4  update_min(offset, ARGS_BUF_SIZE - 1);
5  u32 buffer_index_offset = offset + 1;
6  if (buffer_index_offset > ARGS_BUF_SIZE)
7      return 0;
8  buf->args[offset] = index;
9  // The omitted block clamps size for the older verifier.
10 u32 buffer_arg_offset = buffer_index_offset + size;
11 if (buffer_arg_offset > ARGS_BUF_SIZE)
12     return 0;
13 if (bpf_probe_read(&(buf->args[buffer_index_offset]), size, ptr) < 0)
14     return 0;
15 buf->offset = buffer_arg_offset;
16 buf->argnum++;
17 return 1;
```

Listing 4.8: Checked argument-buffer offset handling.

The current helper checks the original offset and uses `update_min` only to

make its upper bound visible to the verifier. It advances the stored offset only after the index and complete payload have been copied successfully.

Once the event is complete, the common submission helper refreshes its task context, optionally adds a user-space stack identifier and calculates the actual record size. It then sends the record through `BPF_perf_event_output` to the perf-event buffer associated with the current CPU. The Go runtime consumes these per-CPU buffers through a libbpfgo perf reader; the current implementation does not retain a BPF ring-buffer transport path.

Transport loss and decoding failure remain separate conditions. The reader's loss channel reports records dropped because a per-CPU buffer could not retain them, whereas malformed records that reach user space are reported by the decoder. Section 3.4.2 discusses the capacity and ordering consequences of this transport model.

4.4 Kernel-Side UID Filtering

The shared event initialiser implements an optional exact effective UID filter. When the filter is enabled, the program obtains the current task's effective UID and compares it with the configured value. A mismatch terminates the hook invocation before complete context enrichment and perf-buffer submission. Placing this check in the common initialisation path applies the same admission rule to every producer that uses the framework from Section 4.2.1.

This mechanism is deliberately narrow. It accepts one effective UID through an equality comparison and does not implement ranges, exclusion sets, namespace-aware identities or policy expressions. A missing configuration entry or a disabled flag admits the event, while a rejected event creates no correlation state and no authorisation result. Richer event selection remains a user-space responsibility.

The Go runtime writes the enable flag and UID into fixed fields of the shared configuration map before event processing begins. These fields form a binary contract with the C configuration structure and must therefore evolve together. Chapter 5 owns the runtime configuration and map-update lifecycle; this section owns only the kernel-side admission behaviour.

4.5 Compatibility and Verifier Challenges

CO-RE adapts field accesses when compatible BTF information is available, but it does not remove every difference between kernel versions. The shared helpers therefore contain explicit alternatives for task identifiers and start times, capability layouts and inode change times. Cgroup identification also includes a dedicated path for the internal layout used by the RHEL 8 kernel family. At load time, CO-RE selects only branches whose required fields exist on the target kernel.

Some compatibility paths depend on more than structure layout. Syscall registers are read through BTF-aware task helpers when the kernel provides them. Otherwise, an x86-64-specific fallback derives the register frame from the task stack using the page and stack dimensions expected on the target system. Compatible 32-bit processes are detected separately and their syscall numbers are translated before event classification. The source also retains a bounded red-black-tree VMA lookup for kernels that use that representation. It is not called by the memory hooks described in Section 4.2.5 and therefore does not extend the evidence emitted by those events.

The verifier introduces a second class of constraints. Path reconstruction, string-array capture and VMA traversal use fixed loop bounds. Buffer writes and submitted record sizes repeat explicit checks in forms the target verifier can prove safe. The shared syscall-exit path also limits the return-argument index to values below 16, because the broader generic bound was rejected on the target Rocky Linux kernel. Selected memory syscalls use dedicated typed tracepoints instead of the more complex generic dispatcher for the same reason.

These measures allow one eBPF object to accommodate several compatible kernel layouts while remaining acceptable to the verified target. They cannot create missing symbols, hooks, helpers or attachment mechanisms. The portability claim is therefore limited to the identified Rocky Linux environment, rather than to arbitrary Linux versions, consistently with the CO-RE boundaries introduced in Section 2.2.5.

4.6 Kernel-Space Implementation Summary

The kernel-space implementation combines direct, paired and lifecycle producers within one bounded event-construction framework. The hook families

preserve the distinction between process requests, security mediation, committed transitions and returned outcomes, while the indexed argument contract allows unavailable fields to be omitted without changing their schema identity. Records cross the kernel boundary through the perf buffer after optional exact UID admission.

The resulting object is intentionally adapted to the verified Rocky Linux target. CO-RE handles compatible layout differences, while explicit branches and verifier-visible bounds address older and vendor-specific kernel details. Chapter 5 now takes ownership of the user-space runtime that selects and loads these producers, decodes their records and evaluates them through policies and detectors.

Chapter 5

User-Space Runtime and Detection Implementation

Chapter 4 described how the eBPF programs construct and submit binary event records through the perf buffer. This chapter starts at the corresponding user-space boundary. It explains how the Go runtime turns the selected monitoring configuration into a managed user-space pipeline, from program preparation and record interpretation to policy-driven analysis, detector evaluation, and the delivery of monitoring and diagnostic results.

5.1 Command-Line Configuration and Runtime Bootstrap

The user-space workflow begins at the command-line interface, implemented with the Cobra library for Go [31]. Its purpose is to translate operator input into one configuration object shared by the subsequent runtime components. Cobra is responsible only for command definition, argument parsing and invocation of the application entry point; it does not load policies, evaluate detectors or attach eBPF programs.

The configuration is initialised with defaults suitable for the target environment. These include the host BTF path, JSON event and alert output, console-formatted operational logs and an informational log level. CLI flags then update the corresponding fields for event selection, kernel-side filtering, policy and detector paths, output behaviour, logging and eBPF input

selection. Keeping these values in one typed structure avoids passing an independent collection of command-line values through the runtime.

Before any kernel-facing resource is acquired, the configuration passes through normalisation and static validation. Normalisation derives internal settings from the operator’s intent. Supplying one or more detector paths automatically enables detector processing, while selecting **alerts-only** also enables alert generation. Validation then rejects unsupported event-output, alert-output and logging formats. This ordering prevents malformed configuration from reaching eBPF loading or attachment.

The command also separates informational execution from active monitoring. The **list-events** option obtains the public event names from the probe registry and exits without resolving BTF, opening the eBPF object or attaching programs. A normal monitoring invocation instead resolves the eBPF object, constructs the application runner and transfers ownership of the validated configuration to it.

Listing 5.1 summarises this control flow.

```
1  if err := ValidateConfig(&cliCfg); err != nil {
2      return err
3  }
4  if cliCfg.ListEvents {
5      printSupportedEvents(cmd.OutOrStdout())
6      return nil
7  }
8  if err := initialize.BPFObject(&cliCfg); err != nil {
9      return err
10 }
11 runner, err := NewProjectRunner(cliCfg)
12 if err != nil {
13     return err
14 }
15 return runner.Run(ctx)
16
```

Listing 5.1: Validated handoff from the Cobra command to the runtime (abridged).

This staged bootstrap follows a fail-fast design. Errors in static configuration, external definitions or runtime construction are reported before active monitoring begins whenever possible. Record-local failures that occur after startup belong instead to the event-processing lifecycle described later in this chapter.

5.2 Loader, Probe Registry and Attachment Life-cycle

After the command-line layer has produced a validated configuration, the bootstrap process resolves the inputs required by the eBPF runtime. An external object specified through `PROJECT_BPF_OBJECT` has precedence over the command-line path; otherwise, the object embedded in the executable is used. The BTF path follows the corresponding environment, command-line and default host-path order. Embedding simplifies distribution, but loading still requires compatible host BTF, kernel facilities and privileges.

Policy and detector definitions are prepared before the eBPF module is opened. A policy with an explicit event scope can narrow the CLI selection, while an open-ended rule retains the broader scope. If the exact policy scope and CLI selection do not overlap, startup fails instead of interpreting an empty set as a request for all events. Detector selection does not independently enable additional kernel events: its inputs must already belong to the effective monitoring scope.

The probe registry converts the selected logical events into the eBPF programs required for collection. A selected event may also require internal support probes that collect intermediate information without producing a public event.

Listing 5.2 shows how the registry includes both directly selected programs and their internal dependencies.

```
1   for _, probe := range supportedProbes {
2       if probeActivation[probe.eventName] {
3           selectedProbes = append(selectedProbes, probe)
4           continue
5       }
6       for _, event := range probe.impliedBy {
7           if probeActivation[event] {
8               selectedProbes = append(selectedProbes, probe)
9               break
10          }
11      }
12  }
```

Listing 5.2: Selection of producer and support probes (abridged).

Registry tests verify that public events select the expected producers and

support probes. The actual availability of a kernel hook, however, can only be established when the program is loaded and attached on the target system.

Before the eBPF object is loaded, autoload is disabled for registered programs that are not part of the probe plan. Libbpf therefore loads and verifies only the selected producers and their dependencies. This optimisation applies to programs represented in the registry.

After loading, the runtime configures the required maps, creates the perf-event reader and attaches each selected program to its kernel hook. Monitoring continues until the runtime context is cancelled, normally by `SIGINT` or `SIGTERM`.

5.3 User-Space Decoder and Event Registry

Chapter 4 described how the eBPF programs construct and submit binary event records. In user space, the perf reader receives those records as byte slices. The decoder must therefore validate their structure and convert them into the typed representation consumed by the rest of the Go runtime.

Figure 5.1 shows the record regions processed by the decoder. The first 136 bytes contain the common event context. Byte 136 declares the number of arguments, while the remaining bytes contain the variable-length stream of indexed argument records.

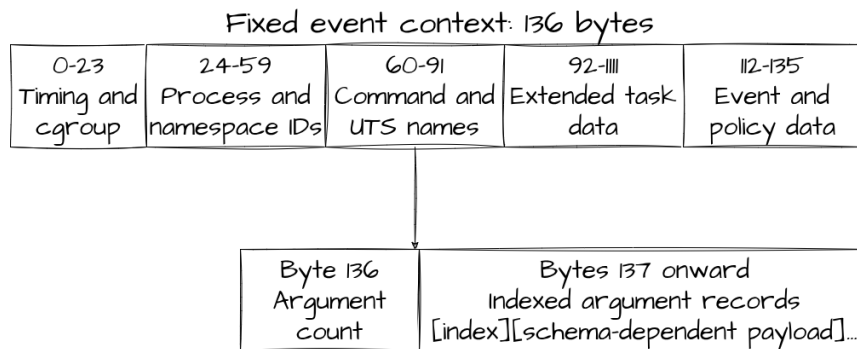


Figure 5.1: Binary record layout consumed by the user-space decoder.

Before reading any field, the decoder verifies that the complete fixed context is available. It then interprets the header through explicit little-endian

offsets. Fields are not obtained by copying the byte region directly into a Go structure, because compiler-defined padding and alignment could make the Go layout differ from the representation produced in C.

The decoded context includes timing and process identity, namespace identifiers, command and UTS names, the numeric event identifier, the associated syscall and other collection metadata. The event identifier is then used to query the user-space event registry.

The registry associates each public event identifier with a logical name and an ordered argument schema. Every schema entry defines the name and expected type of one event-specific argument. This registry is necessary because the wire representation does not include a type tag for each value.

Each argument record begins with a one-byte index followed by its payload. The index selects an entry from the event schema, and the declared type determines how the following bytes must be decoded. Scalar values use fixed widths, whereas strings, byte sequences and arrays carry bounded length information. Socket addresses and credential snapshots use dedicated structured decoders.

```
1 spec, ok := events.GetSpec(eventID)
2 if !ok {
3     return nil, fmt.Errorf("unknown event id=%d", eventID)
4 }
5
6 for seen := 0; seen < argnum && offset < len(rawArgs); seen++ {
7     index := int(rawArgs[offset])
8     offset++
9
10    if index >= len(spec.Args) {
11        return nil, fmt.Errorf("argument index out of range")
12    }
13
14    argSpec := spec.Args[index]
15    value, consumed, err :=
16        decodeArgRecord(rawArgs[offset:], argSpec.DecodeAs)
17    if err != nil {
18        return nil, err
19    }
20
21    offset += consumed
22    decoded[index] = Argument{
23        Name: argSpec.Name,
24        Type: argSpec.DecodeAs,
25        Value: value,
```

```
26     }
27   }
28 }
```

Listing 5.3: Schema-directed decoding of indexed event arguments.

A successfully decoded record becomes a `bufferdecoder.Event`. This structure contains the common context, public logical-event name and typed argument slice. It is the normalised event representation passed to policy matching, detector evaluation and event output; the presentation layer does not perform a separate event normalisation step.

5.4 User-Space Event Selection and Admission

After decoding, the runtime applies three short-circuit admission gates. The first checks whether the decoded logical name belongs to the selected event set. The second applies an optional exact command-name filter. The third asks the policy manager whether the event is selected by the active policy set. A policy evaluates its common scope and then its rules; matching suppress rules override monitor or detect rules. With no configured policies, this final gate is permissive.

Listing 5.4 shows the admission order and the resulting fan-out. Diagnostics and optional socket enrichment are omitted because they do not change the host-event admission contract.

```
1 event, err := bufferdecoder.DecodeEvent(raw)
2 if err != nil {
3     return
4 }
5 if !p.eventEnabled(event.EventName) {
6     return
7 }
8 if !p.commEnabled(event.Context.CommString()) {
9     return
10 }
11 if !p.policyEnabled(event) {
12     return
13 }
14 if !p.cfg.Alerts.Only {
15     if err := p.printer.Print(event); err != nil {
16         p.logger.Error("print event", zap.Error(err))
17     }
18 }
```

```
19 p.processDetectors(ctx, event)
```

Listing 5.4: Decoded-event admission and consumer fan-out (abridged).

An admission failure prevents both normalised-event presentation and detector evaluation.

The policy manager is intentionally not a detector. The current policy modes record monitor, detect or suppress intent, but monitor and detect matches share the same admission path. Suppression is the only mode that removes a matching event from subsequent consumers.

The active loop distinguishes valid rejections from failures. At debug level, only the first few filter drops are logged in order to avoid unbounded diagnostic output. Decode errors, detector errors and rendering errors are logged separately, while the loop remains active. This behaviour is established by code inspection and component tests; a direct end-to-end unit test for the complete admission order is still absent.

5.5 Policy and Detector Engine

5.5.1 YAML Policy and Detector Definitions

Policies and detectors are supplied as separate YAML documents. Both documents are validated and converted into internal Go models before event processing begins.

Listing 5.5 shows the principal policy fields.

```
1  name: privileged-file-monitoring
2  mode: detect
3  intent: threat_hunting
4
5  scope:
6    uids: [0]
7
8  rules:
9    - name: sensitive-files
10      events:
11        include: [security_file_open]
12        filters: ["args.pathname=/etc/*"]
13
14  detectors:
15    include:
```

```
16     severities: [high, critical]
17     techniques: [T1548]
18     stateful: true
19     exclude:
20       tags: [experimental]
21
```

Listing 5.5: Representative YAML policy structure.

The policy `name`, `mode` and `intent` describe its identity and operational purpose. Mode and intent default to `monitor` and `threat_hunting`. The optional `scope` applies command or UID constraints to every rule.

Rules are alternatives, while the fields within one rule are combined as constraints. Event exclusions take precedence over inclusions, and an empty include list initially admits every event. Argument filters use the `args.<name>` form and are evaluated after decoding because they depend on typed event arguments.

The optional `detectors` block selects definitions by identifier, severity, tags, statefulness or MITRE ATT&CK metadata. Values within one criterion are alternatives, whereas different criteria must all match. The exclusion block removes matching definitions from the resulting catalogue. Suppression policies cannot select detectors.

A detector document defines its input events, matching logic, alert metadata and optional ATT&CK classification. Listing 5.6 shows a shortened collective detector.

```
1   id: privilege-exec-chain
2   name: Privilege change followed by execution
3   requires:
4     - {event: security_task_fix_setuid,
5       reason: observes a UID transition}
6     - {event: sched_process_exec,
7       reason: confirms executable replacement}
8   consumes: [security_task_fix_setuid, sched_process_exec]
9
10  stateful: true
11  window: 5s
12  group_by: [process_tree]
13  steps:
14    - event: security_task_fix_setuid
15      conditions:
16        - {field: args.new_euid, operator: eq, value: "0"}
17    - event: sched_process_exec
```

```
18     conditions:
19       - {field: uid, operator: eq, value: "0"}
20
21     output:
22       title: Privilege change followed by process execution
23       severity: high
24     threat:
25       tactics: [{id: TA0004, name: Privilege Escalation}]
26       techniques: [{id: T1548,
27                     name: Abuse Elevation Control Mechanism}]
28     tags: [process, collective, privilege-escalation]
29
```

Listing 5.6: Representative YAML definition of a collective detector.

The stable `id` is used for detector selection and alert attribution. The `requires` entries document why events are needed, while `consumes` defines the set used for event dispatch. If `consumes` is omitted, it is derived from requirements and sequence steps.

Stateless detectors place predicates in a top-level `conditions` list. Collective detectors use ordered `steps`, a bounded `window` and one or more `group_by` strategies. The latter can correlate events by process, immediate process relationship, file resource, local cgroup or a combination of these dimensions. The corresponding correlation strategies and state limits are described in Subsection 5.5.3.

The `output` block defines the generated alert, while `threat` carries optional MITRE ATT&CK classification. Conditions, event names, severity values, ATT&CK identifiers and correlation plans are validated and compiled during startup.

Files may be supplied directly or through directories scanned non-recursively in lexical order.

5.5.2 Point and Collective Evaluation Paths

The conceptual distinction between point and collective detectors was introduced in Subsection 3.6.2. In the implementation, both forms satisfy the same `Detector` interface and enter through the same event-indexed dispatcher. The distinction is made inside the YAML-backed detector when an admitted event is evaluated.

Listing 5.7 shows the branch that separates immediate point evaluation from

stateful sequence processing.

```
1  if !d.consumes(event.EventName) {
2      return nil, nil
3  }
4
5  if d.isStatefulSequence() {
6      return d.onStatefulEvent(event)
7  }
8
9  matches, err := d.matches(event)
10 if err != nil || !matches {
11     return nil, err
12 }
13
14 return []detectors.Alert{d.newAlert(event)}, nil
15
16 func (d *Detector) isStatefulSequence() bool {
17     return d.def.Stateful && len(d.steps) > 1
18 }
19
```

Listing 5.7: Selection of the point or collective evaluation path.

The first check protects the detector boundary even though the dispatcher has already narrowed the candidate set. The second check performs the actual classification: a detector uses the collective path only when it is marked as stateful and contains more than one compiled step. Setting `stateful: true` without a multi-step sequence is therefore not sufficient to create a collective detector.

Collective definitions are delegated to `onStatefulEvent`. This path derives the configured correlation keys, looks for an existing partial sequence and tests the event against its next expected step. It may advance an existing sequence, complete it and emit its ordered evidence, or create a new partial sequence when the first step matches.

5.5.3 Compiled Correlation State and Alert Deduplication

Chapter 3 introduced the local identities used to relate events. In the implementation, the YAML `group_by` values are compiled into a correlation plan when the detector is constructed. Event-time processing therefore executes preselected key extractors rather than repeatedly parsing strategy names.

The supported components produce the following internal values:

- `process` combines the host PID and task start time;
- `process_tree` can additionally produce the immediate parent's host PID and start time when looking up an existing sequence;
- `resource` combines the device and inode arguments;
- `cgroup` uses the local kernel cgroup identifier.

When several components are configured, their values are joined into one composite key. Every component must be available: missing process, resource or cgroup data prevents the event from advancing the sequence instead of causing a fallback to a weaker identity.

Plan construction also validates the selected strategies. Unknown or duplicate components are rejected, and resource correlation is accepted only when every sequence step exposes both `dev` and `inode`. The unsupported `user_session` strategy is rejected because the event model does not provide a stable session identifier; UID is deliberately not used as an approximation.

Each collective detector owns a map from correlation keys to partial sequence state. A state entry stores the next expected step, the time at which the sequence began and the ordered evidence already matched. Listing 5.8 shows how an incoming event advances one of these entries.

```
1   for _, key := range lookupKeys {
2       state, ok := d.state[key]
3       if !ok {
4           continue
5       }
6       if now.Sub(state.firstSeen) > window {
7           delete(d.state, key)
8           continue
9       }
10
11      matches, err :=
12          stepMatches(event, d.steps[state.nextStep])
13      if err != nil {
14          return nil, err
15      }
16      if !matches {
17          continue
18      }
19
20      state.events = append(state.events, event)
21      state.nextStep++
```

```

22
23     if state.nextStep >= len(d.steps) {
24         delete(d.state, key)
25         return []detectors.Alert{
26             d.newAlertFromEvents(state.events),
27         }, nil
28     }
29
30     d.state[key] = state
31     return nil, nil
32 }
33

```

Listing 5.8: Advancement of a partial collective sequence (abridged).

For process-tree correlation, the lookup keys may contain both the current process and its immediate parent. This allows an event generated by a child to advance state previously created for its parent without constructing a persistent process graph. If no existing state advances, an event matching the first step creates a new entry under its initial correlation key. Only one partial sequence is retained for each key.

Table 5.1 collects the limits applied to this state. These values are implementation safeguards and are not performance results.

Mechanism	Value	Runtime effect
Stateful window	2-second default; 5-second maximum	Bounds the lifetime of incomplete evidence. Longer definitions are rejected during validation.
Periodic pruning	$\min(\text{window}/2, 1 \text{ second})$	Avoids scanning the complete state map for every event. The directly addressed entry is still checked immediately.
Sequence capacity	4,096 keys per detector	Inserting a new key at capacity evicts the oldest incomplete sequence.
Alert deduplication	5-second suppression window	Prevents repeated equivalent alerts from being emitted during the configured interval. Entries older than 10 seconds are removed when later alert batches are processed.

Table 5.1: Runtime bounds for collective state and alert deduplication.

Expired state is removed when its key is addressed and through periodic pruning triggered by relevant event processing. Although the detector interface exposes a `Flush` operation, the production runtime does not schedule it with an independent timer. Completely idle stale entries may therefore remain allocated until later relevant traffic or capacity pressure causes cleanup.

Alert deduplication runs after detector evaluation and is separate from sequence correlation. Its key contains detector identity, title, severity and the fields of the first evidence event, while timestamps and thread identifiers are excluded. A repeated key inside the suppression window is not emitted.

This mechanism reduces repeated alert output, but it does not represent the complete collective sequence or a durable causal identity. The deduplication map also has no independent capacity limit, which remains an implementation constraint to consider during evaluation.

5.6 Output and Structured Logging

The output package is a presentation boundary, not another normalisation stage. It adapts the decoded event into JSON or compact text and adapts a detector alert into a user-facing record containing detector identity, severity, ordered evidence and optional threat metadata. JSON preserves typed values and may add readable labels, whereas compact text is intended for interactive inspection. The event model used by output is derived from the same decoded representation used by policies and detectors.

Normalised event output and alert output both use standard output. They are configured independently and can therefore use different formats in one run; the resulting stream is not guaranteed to be one homogeneous machine-readable schema. The `alerts-only` option suppresses normalised event output but leaves detector evaluation and alert rendering active. Point alerts expose one source event, while collective alerts retain their ordered evidence in JSON and render a bounded sequence in compact text.

Operational diagnostics follow a separate path. The Zap logger writes to standard error in console or JSON form, with debug, info, warn and error levels. It receives runtime lifecycle messages, selected libbpf diagnostics, perf-buffer loss notifications, decoding failures, detector failures and presentation failures [64]. Zap does not serialise monitored events or alerts. This prevents infrastructure errors from being mistaken for security observations and allows diagnostics to change without changing the data model seen by

event and alert consumers.

Table 5.2 summarises the operating channels and error boundaries. Fatal configuration and startup failures are returned to the command entry point, while most record-local failures are logged and the loop continues. Logger synchronisation errors are currently discarded, so the design does not claim durable delivery of every diagnostic message.

Class	Source model	Channel	Failure behaviour
Normalised event	<code>bufferdecoder.Event</code> through a printer	stdout	A write failure is diagnosed; subsequent events continue.
Detector alert	Detector alert with one event or ordered evidence	stdout	A rendering failure is diagnosed; it does not change detector state.
Operational diagnostic	Runtime, decoder, detector, output or libbpf condition	stderr through Zap	Record-local conditions are logged without becoming security events.
Fatal bootstrap or lifecycle error	Command, definition loading or runtime setup return value	stderr through the command path	Startup or shutdown reports an error; cleanup errors are joined where available.

Table 5.2: User-space output and diagnostic boundaries.

5.7 User-Space Implementation Summary

The user-space implementation turns a selected perf-buffer record into a typed event, applies explicit admission gates, and then routes admitted evidence to presentation and deterministic detection. Policies control scope and suppression, while detectors evaluate point conditions or bounded local sequences and attach ATT&CK classification metadata to their alerts. The runtime keeps kernel-facing resource ownership, record interpretation, analytical state and presentation responsibilities separate.

The implementation also defines clear boundaries for the following evaluation. The registry and decoder tests establish important structural invariants, but attachment behaviour, output under workload, alert quality, loss and resource consumption require target-environment evidence. Chapter 6 evaluates these behaviours on the stated Rocky Linux environment.

Chapter 6

Experimental Evaluation

Chapters 4 and 5 described the kernel-space collection path and the user-space runtime as separate implementation concerns. This chapter evaluates their integrated behaviour in a controlled container environment. The experiment follows one scenario from workload execution to the evidence observed by the monitor, the alert produced by the collective detector and the resources charged to the monitored containers. The purpose is to determine whether the implemented pipeline behaves as intended under a reproducible node-local workload.

6.1 Evaluation Objectives and Experimental Protocol

The evaluation addresses three questions.

- Can the monitoring agent observe security-relevant activity generated inside another container when the Pod does not share its process namespace with the host?
- Can the configured collective detector assemble the expected sequence of process and file events while leaving a controlled benign interaction without an alert?
- How does exact kernel-side UID filtering affect the resources charged to the monitor container during the attack workload?

Two campaigns were conducted. The functional campaign comprised three benign windows and four exploit windows. Each window had a distinct run identifier and preserved the trigger output, standard error and time-bounded container logs. The performance campaign comprised five rounds, each containing a filtered monitor profile, an unfiltered monitor profile and a profile without the monitor. The starting profile was shifted between rounds, giving fifteen runs in a cyclic order. The Pod was recreated and allowed to become ready before each run.

The performance protocol used a ten-second warm-up and then selected samples whose endpoint timestamps fell within the externally recorded workload interval. The sampling period was nominally one second, although reads performed through the Minikube SSH path introduced some interval variation. A project acceptance gate of 5% was applied to the mean CPU time charged to the monitor cgroup, expressed as a percentage of one core. This gate is local to the project and is not a claim about total host overhead. Results are reported descriptively using the five run means per profile; the sample standard deviation describes variation among those runs rather than a wider population.

The experimental input was the scenario executed by the trigger together with the selected monitoring profile. The main dependent observations were the event evidence and alert count in the functional campaign, and cgroup CPU, memory, RSS, PID count, node CPU and workload duration in the performance campaign. The protocol was designed to hold the host, cluster allocation, target images, policy, detector and synthetic flag fixed. Run identifiers and UTC window boundaries tied the scenario output to the corresponding monitor log, preventing alerts from earlier activity in the accumulated container log from being counted in a later run.

6.2 Experimental Testbed

Laboratory records identify the host as Rocky Linux 8.10 with kernel `4.18.0-553.137.1.el8_10.x86_64`. This is a later patch-level build in the same Rocky Linux 8.10 kernel stream as the implementation baseline used during development. The recorded Minikube v1.38.1 configuration created one node with the Docker driver, two requested CPUs and 4096 MiB of memory [58]. The archived Kubernetes output independently confirms server version v1.30.0. No evaluation claim depends on functionality introduced after the implementation baseline.

The testbed used a single Pod named `aes-beta-lab` containing three containers. The target exposed the vulnerable service, the trigger generated benign or exploit inputs, and the monitor ran the proposed tool. The trigger reached the target through the Pod-local loopback address on port 1337. The applied Pod set `hostPID`, `hostNetwork` and `shareProcessNamespace` to false. The target and trigger consequently shared the Pod network but did not share one process namespace [38, 39].

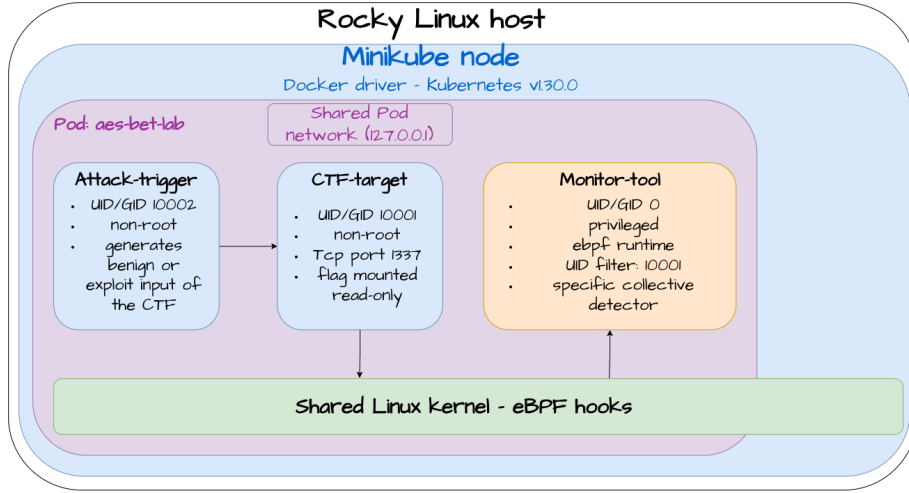


Figure 6.1: Experimental topology of the three-container Pod and its relationship with the shared node kernel.

Figure 6.1 provides the corresponding configuration details. The target ran as UID and group identifier 10001, while the trigger used 10002. Both were non-root, had privilege escalation disabled, used read-only root filesystems and dropped all Linux capabilities. The monitor ran as root in a privileged container with an unconfined seccomp profile. It mounted host BTF read-only and the host BPF and tracing filesystems read-write. These permissions were required by the laboratory configuration and must not be interpreted as a least-privilege deployment result [37]. The default service-account token also remained mounted, although no evaluated scenario depended on Kubernetes API access.

Dimension	Experimental configuration
Host	Rocky Linux 8.10; kernel recorded as 4.18.0-553.137.1.el8_10.x86_64
Cluster	Minikube v1.38.1; Docker driver; Kubernetes v1.30.0; one node with two requested CPUs and 4096 MiB
Pod isolation	Three containers; <code>hostPID: false</code> ; <code>hostNetwork: false</code> ; <code>shareProcessNamespace: false</code>
Target and trigger	Non-root UIDs 10001 and 10002; read-only root filesystems; no privilege escalation; all capabilities dropped
Monitor	Privileged UID 0; unconfined seccomp; host BTF, bpffs and tracefs mounted into the container
Monitored event set	<code>sched_process_exec</code> , <code>security_file_open</code> and <code>open</code>
Workload provenance	AES-beta from openECSC-2024; recorded revision 2602f22c; checked binary, loader and C library; synthetic flag

Table 6.1: Experimental environment and controlled configuration.

The public challenge revision was recorded before image construction, and the binary, dynamic loader and C library were checked against fixed hashes. The preparation script verifies these artefacts but does not independently assert the Git revision of the source checkout. The applied Pod and the policy and detector documents were preserved with configuration hashes for the functional campaign, while the running Pod record retained the three image identities. The flag was a synthetic value created for the thesis experiment rather than a competition secret. Full hashes, image identifiers, manifests and commands remain in the reproducibility material rather than in the chapter body.

The Pod specifications declared resource requests but no container limits. Consequently, the measurements describe actual use under contention on the two-core Minikube node rather than enforcement at a fixed container ceiling. The requested node size is preserved by the cluster-start command; a separate snapshot of realised Kubernetes allocatable capacity was not archived. This preservation gap is considered again in the validity analysis.

6.3 Controlled AES-beta Scenario

AES-beta is a public Capture the Flag challenge adapted here as a controlled process-monitoring workload. The target service receives requests over the Pod-local connection. A benign run exercises the service without attempting to read the mounted flag. An exploit run drives the vulnerable service until the target process invokes a shell, executes `cat` and attempts to read the flag at `/opt/aes-beta/flag`. The exploit implementation and payload construction are not evaluation variables; the relevant input is the resulting host behaviour [17].

The policy admitted only the three logical events required by the experiment. The detector then required four ordered observations for UID 10001 within a five-second window. The first two were successful executable transitions to `/bin/sh` and `/bin/cat`. The third was `security_file_open` for the resolved flag path. This event shows that the resolved file reached the Linux security mediation stage, but it does not by itself establish that access succeeded. The fourth was an `open` event for the caller pathname `flag` with a non-negative return value, which supplied the syscall-boundary success evidence.

The sequence used the local `process_tree` correlation strategy. In the observed alerts, all four records shared one host PID; the experiment therefore exercised the same-process branch of that strategy. It did not independently test correlation from a parent process to its immediate child. The resulting alert carried high severity and the ATT&CK identifiers TA0002, TA0009, T1059.004 and T1005. These identifiers classify the intended meaning of the observed sequence in the standard MITRE ATT&CK vocabulary [29]; they are not evidence of detector correctness or technique coverage.

This scenario creates an interpretable boundary between controlled application input and observable host effects. Unlike direct invocation of the monitored commands, it requires the evidence to originate from the vulnerable target service rather than from the trigger container. The challenge artefacts and runtime dependencies were fixed for the experiment, and the success condition used a synthetic value whose disclosure has no operational consequence.

6.4 Functional Evaluation

6.4.1 Cross-Container Event Visibility

The initial visibility test captured a wider stream of 7,289 records and then isolated the nine records whose process UID was 10001. Those records described a directly invoked `cat` operation in the target container, including the executable transition, resolved flag object and successful file-open result. The process had container PID 327 and host PID 3016236. The monitor therefore retained both the namespace-relative identity reported inside the target and the host identity needed for node-local correlation.

This result demonstrates that the privileged monitor observed target-container activity despite `hostPID: false` and without a shared Pod process namespace. The scope is deliberately narrow: all containers ran on one Minikube node and shared its kernel. The result does not establish visibility across nodes or cluster-wide correlation.

6.4.2 Collective Detector Validation

Every exploit window produced one alert containing the configured four-event sequence. In the first run, the two executable transitions identified `/bin/sh` and `/bin/cat`; the mediation event then identified the resolved flag object, and the final `open` event returned descriptor 2. The latter supplied the syscall-boundary success evidence that the LSM event alone could not provide.

The first and last event timestamps were separated by 0.941, 0.625, 0.650 and 0.725 ms in the four exploit windows. These intervals show that the four observations fell well inside the configured five-second sequence window. They must not be interpreted as detector latency or end-to-end alert-delivery time, because they measure only the span between kernel event timestamps. Each alert also retained a single unique host PID across its evidence, confirming the local same-process correlation used in this scenario.

The monitor and the exploit client observed related but distinct outcomes. The alert demonstrates that the shell and `cat` transitions occurred and that the flag file was successfully opened. It does not demonstrate that the file contents were returned through the service and received by the trigger. This distinction became visible in the third exploit run, where the detector completed the same four-event sequence but the client received an empty

flag value and exited with status 1.

6.4.3 Benign Control and Repeated Runs

Table 6.2 reports the seven time-bounded functional windows. The three benign interactions completed successfully and produced no detector alert. The four exploit interactions each produced one alert with four evidence events. Three returned the synthetic flag to the client; E3 did not. No matching perf-loss diagnostic appeared in any selected window. This means only that the implemented diagnostic path reported no loss condition; it is not proof that transport was loss-free.

Run	Scenario outcome	Internal (ms)	Wall (ms)	Alerts	Loss diag.
B1	Benign pass	17	1,118	0	0
B2	Benign pass	11	1,024	0	0
B3	Benign pass	11	1,038	0	0
E1	Exploit pass; flag returned	48,543	49,627	1	0
E2	Exploit pass; flag returned	51,584	52,759	1	0
E3	Empty client output; status 1	49,509	50,591	1	0
E4	Diagnostic exploit pass; flag returned	49,548	50,754	1	0

Table 6.2: Results of the repeated functional windows. Every exploit alert contained four ordered evidence events.

The observations support a bounded functional conclusion: the implemented pipeline collected the selected target-container events and completed the configured collective sequence in these four controlled exploit windows, while the three selected benign interactions did not complete it. They do not support a detection-accuracy percentage. The detector was constructed for this known scenario, the benign workload was narrow and no independent holdout set was evaluated.

6.5 Performance Evaluation

6.5.1 Metrics, Sampling and Benchmark Profiles

The filtered profile ran the monitor with the exact UID 10001 kernel filter. The unfiltered profile used the same policy, detector, alert mode and workload without that filter. The no-monitor profile removed the monitor container and served only as context for the target workload. Before every run, the Pod was deleted, recreated under the selected profile and allowed to reach readiness. The order was rotated over five rounds rather than randomised.

The filtered and unfiltered profiles preserved the same event policy, detector, alert-only output and exploit workload. Their intended independent variable was therefore the exact kernel-side UID admission setting. The no-monitor profile changed a broader part of the Pod by removing the monitor container and its host mounts. It was retained to show the scale and variability of the target workload, not as an otherwise identical control for causal overhead estimation.

Container CPU was calculated from `cgroup-v1 cpuacct.usage` deltas, which report accumulated cgroup CPU time in nanoseconds, divided by wall-clock deltas [47]. A value of 100% represents one fully used core. Node CPU was derived from `/proc/stat` and used the complete two-core node as its 0–100% scale. Cgroup memory usage and resident set size (RSS) were collected separately because they describe different boundaries: cgroup memory can include charged pages beyond the resident process set. PID counts and the workload’s internal duration were also preserved.

For filtered and unfiltered runs, the primary cgroup belonged to the monitor and the target was sampled as a secondary container. In the no-monitor profile, the primary cgroup necessarily belonged to the target. Its CPU and memory columns therefore cannot be compared directly with the monitor columns or subtracted as a baseline for monitor overhead.

6.5.2 Resource Consumption and Workload Duration

Table 6.3 contains the replicated summary. Each CPU mean and standard deviation was calculated over five per-run means. The filtered monitor used 0.39% of one core on average, with a sample standard deviation of 0.02 percentage points. The unfiltered monitor used 4.87% on average with a

standard deviation of 0.11. The mean of each run’s 95th percentile was 0.92% for the filtered monitor and 7.72% for the unfiltered monitor; the largest individual samples were 1.32% and 9.27%, respectively.

Profile and metric owner	CPU mean $\pm$ SD	Median; range	Campaign max memory / RSS	Workload duration mean $\pm$ SD
Filtered monitor	0.39% $\pm$ 0.02	0.39%; 0.36–0.41%	100.51 / 39.36 MiB	54.85 $\pm$ 4.09 s
Unfiltered monitor	4.87% $\pm$ 0.11	4.89%; 4.73–4.98%	112.16 / 46.02 MiB	61.33 $\pm$ 3.04 s
No-monitor target	34.80% $\pm$ 3.57	35.90%; 28.66–37.74%	19.02 / 6.88 MiB	56.27 $\pm$ 7.22 s

Table 6.3: Replicated performance results, with five runs per profile. CPU and duration values summarise per-run observations; memory and RSS are the maxima across all selected samples and replicas. Metrics belong to the container named in the first column.

The maximum cgroup memory attributed to the monitor was 100.51 MiB when filtered and 112.16 MiB when unfiltered; maximum RSS was 39.36 and 46.02 MiB. These maxima describe the resources observed in this campaign rather than a steady-state requirement. The no-monitor memory values belong to the target and are included only to characterise the workload context.

Mean node CPU remained close to saturation in every profile: 94.70% for filtered, 95.11% for unfiltered and 94.89% without the monitor. Workload duration also varied among repetitions. Although the filtered mean duration was shorter than the unfiltered mean, the small sample, changing workload cost and high node utilisation prevent a causal interpretation. The experiment therefore does not use duration differences to estimate monitor overhead.

6.5.3 Observed Difference with Kernel-Side UID Filtering

All five filtered monitor run means fell below the project-defined 5% gate, ranging from 0.36% to 0.41% of one core. The five unfiltered means ranged from 4.73% to 4.98%. Under the scripted profiles, enabling exact UID admission was associated with a reduction from 4.87% to 0.39% of one core, or approximately 92.0% relative to the unfiltered mean. Both profiles were

repeated in every round, but the evidence archive does not retain the rendered monitor configuration and image identity for each individual replica. The comparison therefore supports a strong association under the intended profiles rather than proving that the filter was the sole cause of the difference.

The result remains narrower than a total-overhead statement. Work executed by eBPF programs in kernel context is not fully represented by the monitor container’s cgroup charge, and the experiment did not isolate host-level kernel cost. The 5% result therefore applies to measured monitor-cgroup CPU in the filtered profile, not to the complete system cost of monitoring.

The functional path remained active in the performance campaign. Each of the ten monitored runs produced exactly one alert, and none contained a matching textual perf-loss diagnostic. This supports the claim that filtering preserved the configured scenario evidence in the observed runs. It does not demonstrate loss-free transport, because absence of a diagnostic is not an independent accounting mechanism for every generated record.

6.6 Discussion of Results

The functional and performance campaigns support three complementary findings. First, the monitor observed process and file activity produced in the target container while host PID sharing was disabled at the Pod level. Host process identity allowed the evidence to be related locally without presenting the container PID as a global identity. Second, the detector retained the semantic distinction between executable-transition evidence, file-security mediation and the successful file-open result. This produced one ordered alert in every exploit window and avoided interpreting the LSM observation alone as a completed access.

Third, the scripted profile with early UID admission was associated with substantially less work charged to the monitor cgroup. The approximately 92.0% difference is consistent with the intended benefit of rejecting unrelated events before transport and user-space processing. However, incomplete per-run configuration provenance and near-saturated node utilisation prevent a stronger causal claim or an extension to total overhead and workload acceleration.

The divergent third exploit run also clarifies the monitor’s evidence bound-

ary. The alert correctly represented the host operations it was designed to observe, including a successful file descriptor return. The empty client result belonged to a later application and communication outcome outside the detector sequence. This is not a contradiction between client and monitor; it demonstrates why observed kernel evidence and end-to-end application success must be reported separately.

6.7 Threats to Validity and Current Limitations

Internal validity. The performance profiles were rotated cyclically rather than randomly assigned. Recreating the Pod reduced carry-over of container state, but did not remove host load, cache effects or variation in exploit execution. Sampling through the Minikube SSH path produced intervals longer than the nominal one second. Because a sample was selected by its endpoint timestamp while its CPU value covered the preceding interval, the first selected sample could include some pre-workload activity and the final partial interval could be omitted. The node was close to saturation in all profiles. In the functional campaign, E3 also showed that client output could fail despite completion of the monitored host sequence.

Construct validity. CPU charged to the monitor cgroup is not equivalent to total monitoring overhead, and target metrics from the no-monitor profile are not a monitor baseline. Cgroup memory and RSS cover different resources. Event timestamp spans do not measure detector or output latency. Likewise, the absence of a loss diagnostic does not prove loss-free perf-buffer transport. The functional interpretation also depends on preserving the difference between an LSM mediation event and the successful syscall return.

External validity. The experiment used one Rocky Linux kernel build, one Docker-backed Minikube node, one Kubernetes version, one public CTF challenge and one detector tailored to that challenge. The benign interaction represented only a narrow control. No alternative kernels, long-duration workloads, multi-node deployments or production workload mixtures were evaluated. The results therefore describe the tested node-local scenario and should not be generalised to arbitrary container environments.

Conclusion validity. Five repetitions per performance profile and seven functional windows support descriptive comparisons, not inferential statis-

tics or accuracy rates. The approximately 92.0% profile difference applies to the selected event scope, UID and workload on the observed node. The per-run performance archive does not retain every rendered monitor configuration and image identity, so it cannot independently establish that the UID filter was the only realised difference. Differences in target CPU, memory or duration likewise do not establish a causal performance effect. ATT&CK metadata provides alert classification and does not validate detector coverage.

6.8 Evaluation Summary

The experiment demonstrated node-local observation of target-container process and file activity, completion of the configured four-event collective sequence in four controlled exploit windows, and no alert in three selected benign windows. Under the scripted performance profiles, exact UID filtering was associated with an approximately 92.0% lower mean CPU charge than the unfiltered profile, with all five filtered run means below the local 5% gate. These findings validate the intended workflow within the stated environment while leaving total overhead, broad detection accuracy and multi-node behaviour outside the supported claims. Chapter 7 uses these bounded results to draw the final conclusions and identify future work.

Appendix A

Appendix A

Appendix text here.

Appendix B

Appendix B

Appendix text here.

Bibliography

- [1] Jagrati Agrawal et al. “Efficient Pattern Matching over Event Streams”. In: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*. Ed. by Jason Tsong-Li Wang. New York, NY, USA: Association for Computing Machinery, 2008, pp. 147–160. ISBN: 978-1-60558-102-6. DOI: [10.1145/1376616.1376634](https://doi.org/10.1145/1376616.1376634).
- [2] Aqua Security. *Installing Tracee*. Official Tracee documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/install/> (visited on 07/29/2026).
- [3] Aqua Security. *libbpfgo: An eBPF Library for Go Powered by libbpf*. Official project repository. 2026. URL: <https://github.com/aquasecurity/libbpfgo> (visited on 07/27/2026).
- [4] Aqua Security. *Tracee Architecture*. Official architecture documentation for Tracee version 0.15. 2023. URL: <https://aquasecurity.github.io/tracee/v0.15/contributing/architecture/> (visited on 07/29/2026).
- [5] Aqua Security. *Tracee Custom Events*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/events/custom/overview/> (visited on 07/29/2026).
- [6] Aqua Security. *Tracee Detector API Reference*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/detectors/api-reference/> (visited on 07/27/2026).
- [7] Aqua Security. *Tracee Detectors*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/detectors/> (visited on 07/27/2026).
- [8] Aqua Security. *Tracee Documentation*. 2026. URL: <https://aquasecurity.github.io/tracee/latest/> (visited on 07/17/2026).

BIBLIOGRAPHY

- [9] Aqua Security. *Tracee Documentation Overview*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/overview/> (visited on 07/29/2026).
- [10] Aqua Security. *Tracee Events*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/events/> (visited on 07/27/2026).
- [11] Aqua Security. *Tracee Output Configuration*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/outputs/> (visited on 07/27/2026).
- [12] Aqua Security. *Tracee Policies: Overview*. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/policies/> (visited on 07/17/2026).
- [13] Aqua Security. *Tracee Policy Rules*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/policies/rules/> (visited on 07/29/2026).
- [14] Aqua Security. *Tracee Security Model*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/security-model/> (visited on 07/29/2026).
- [15] Varun Chandola, Arindam Banerjee, and Vipin Kumar. “Anomaly Detection: A Survey”. In: *ACM Computing Surveys* 41.3 (July 2009), 15:1–15:58. DOI: [10.1145/1541880.1541882](https://doi.org/10.1145/1541880.1541882).
- [16] Gianpaolo Cugola and Alessandro Margara. “Processing Flows of Information: From Data Stream to Complex Event Processing”. In: *ACM Computing Surveys* 44.3 (June 2012), 15:1–15:62. DOI: [10.1145/2187671.2187677](https://doi.org/10.1145/2187671.2187677).
- [17] ECSC2024. *AES-beta Challenge Source*. Official openECSC-2024 challenge repository. 2024. URL: <https://github.com/ECSC2024/openECSC-2024/tree/main/teamitaly-preselection-ctf/pwn01> (visited on 08/13/2026).
- [18] Hojat Gazestani. *Understanding Linux Process UIDs: Real, Effective, Saved, and Filesystem*. Sept. 2, 2025. URL: <https://hojat-gazestani.github.io/blogs/Linux-Standard-UNIX-Access-Control> (visited on 08/04/2026).
- [19] Bolaji Gbadamosi et al. “The eBPF Runtime in the Linux Kernel”. In: *arXiv preprint arXiv:2410.00026* (2024). DOI: [10.48550/arXiv.2410.00026](https://doi.org/10.48550/arXiv.2410.00026). URL: <https://arxiv.org/abs/2410.00026>.

BIBLIOGRAPHY

- [20] Elazar Gershuni et al. “Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions”. In: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. Ed. by Kathryn S. McKinley and Kathleen Fisher. PLDI ’19. Association for Computing Machinery, 2019, pp. 1069–1084. doi: [10.1145/3314221.3314590](https://doi.org/10.1145/3314221.3314590). URL: <https://doi.org/10.1145/3314221.3314590>.
- [21] Yi He et al. “Cross Container Attacks: The Bewildered eBPF on Clouds”. In: *32nd USENIX Security Symposium (USENIX Security 23)*. Ed. by Joseph A. Calandrino and Carmela Troncoso. Anaheim, CA: USENIX Association, Aug. 2023, pp. 5971–5988. ISBN: 978-1-939133-37-3. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/he>.
- [22] Jin Her et al. “An In-Depth Analysis of eBPF-Based System Security Tools in Cloud-Native Environments”. In: *IEEE Access* 13 (2025), pp. 155588–155604. doi: [10.1109/ACCESS.2025.3605432](https://doi.org/10.1109/ACCESS.2025.3605432). URL: <https://ieeexplore.ieee.org/document/11146725/>.
- [23] Linux Kernel Community. *eBPF Userspace API*. Linux kernel 6.9 documentation. 2024. URL: <https://www.kernel.org/doc/html/v6.9/userspace-api/ebpf/index.html> (visited on 07/17/2026).
- [24] Linux Kernel Community. *eBPF Verifier*. 2026. URL: <https://docs.kernel.org/bpf/verifier.html> (visited on 07/17/2026).
- [25] Linux man-pages project. *bpf-helpers(7) — List of eBPF Helper Functions*. 2026. URL: <https://man7.org/linux/man-pages/man7/bpf-helpers.7.html> (visited on 07/27/2026).
- [26] Linux man-pages project. *bpf(2) — Perform a Command on an Extended BPF Map or Program*. 2026. URL: <https://man7.org/linux/man-pages/man2/bpf.2.html> (visited on 07/27/2026).
- [27] Linux man-pages project. *inode(7) — Linux Manual Page*. 2026. URL: <https://man7.org/linux/man-pages/man7/inode.7.html> (visited on 08/06/2026).
- [28] Red Hat. *What Is Backporting and How Does It Affect Red Hat Enterprise Linux?* Red Hat Customer Portal. June 2025. URL: <https://access.redhat.com/solutions/57665> (visited on 07/27/2026).

BIBLIOGRAPHY

- [29] Blake E. Strom et al. *MITRE ATT&CK: Design and Philosophy*. Tech. rep. The MITRE Corporation, Mar. 2020. URL: <https://www.mitre.org/news-insights/publication/mitre-attck-design-and-philosophy>.
- [30] Dave Thaler. *BPF Instruction Set Architecture (ISA)*. RFC 9669. Internet Engineering Task Force, Oct. 2024. DOI: [10.17487/RFC9669](https://doi.org/10.17487/RFC9669). URL: <https://www.rfc-editor.org/info/rfc9669>.
- [31] The Cobra Authors. *Cobra: A Commander for Modern Go CLI Interactions*. Official project repository. 2026. URL: <https://github.com/spf13/cobra> (visited on 08/10/2026).
- [32] The Falco Project Authors. *Falco Event Sources*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/event-sources/> (visited on 07/27/2026).
- [33] The Falco Project Authors. *Falco Kernel Event Architecture*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/event-sources/kernel/architecture/> (visited on 07/27/2026).
- [34] The Falco Project Authors. *Falco Outputs*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/outputs/> (visited on 07/27/2026).
- [35] The Falco Project Authors. *Falco Rule Conditions*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/rules/conditions/> (visited on 07/27/2026).
- [36] The Falco Project Authors. *Falco Rules*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/rules/> (visited on 07/27/2026).
- [37] The Kubernetes Authors. *Linux Kernel Security Constraints for Pods and Containers*. Official Kubernetes documentation. 2024. URL: <https://kubernetes.io/docs/concepts/security/linux-kernel-security-constraints/> (visited on 08/13/2026).
- [38] The Kubernetes Authors. *Services, Load Balancing, and Networking*. Official Kubernetes documentation. 2026. URL: <https://kubernetes.io/docs/concepts/services-networking/> (visited on 08/13/2026).
- [39] The Kubernetes Authors. *Share Process Namespace between Containers in a Pod*. Official Kubernetes documentation. 2024. URL: <https://kubernetes.io/docs/tasks/configure-pod-container/share-process-namespace/> (visited on 08/13/2026).

BIBLIOGRAPHY

- [40] The Linux Kernel Documentation. *BPF Design Q&A*. 2026. URL: https://docs.kernel.org/bpf/bpf_design_QA.html (visited on 07/27/2026).
- [41] The Linux Kernel Documentation. *BPF LLVM Relocations*. 2026. URL: https://docs.kernel.org/bpf/llvm_reloc.html (visited on 07/27/2026).
- [42] The Linux Kernel Documentation. *BPF Maps*. 2026. URL: <https://docs.kernel.org/bpf/maps.html> (visited on 07/27/2026).
- [43] The Linux Kernel Documentation. *BPF Ring Buffer*. 2026. URL: <https://docs.kernel.org/bpf/ringbuf.html> (visited on 07/27/2026).
- [44] The Linux Kernel Documentation. *BPF Ring Buffer*. 2026. URL: <https://docs.kernel.org/bpf/ringbuf.html> (visited on 08/03/2026).
- [45] The Linux Kernel Documentation. *BPF Type Format (BTF)*. 2026. URL: <https://docs.kernel.org/bpf/btf.html> (visited on 07/27/2026).
- [46] The Linux Kernel Documentation. *Classic BPF vs eBPF*. Describes execution through architecture-specific JIT compilation and, when JIT compilation is not used, through the `__bpf_prog_run()` interpreter. URL: https://docs.kernel.org/bpf/classic_vs_extended.html (visited on 07/27/2026).
- [47] The Linux Kernel Documentation. *CPU Accounting Controller*. 2026. URL: <https://docs.kernel.org/admin-guide/cgroup-v1/cpuacct.html> (visited on 08/13/2026).
- [48] The Linux Kernel Documentation. *Documentation for /proc/sys/net/: BPF JIT Controls*. 2026. URL: <https://docs.kernel.org/admin-guide/sysctl/net.html#bpf-jit-enable> (visited on 07/27/2026).
- [49] The Linux Kernel Documentation. *Kernel Probes (Kprobes)*. 2026. URL: <https://docs.kernel.org/trace/kprobes.html> (visited on 07/27/2026).
- [50] The Linux Kernel Documentation. *libbpf Overview*. 2026. URL: https://docs.kernel.org/bpf/libbpf/libbpf_overview.html (visited on 07/27/2026).
- [51] The Linux Kernel Documentation. *Linux Security Module Development*. 2026. URL: <https://docs.kernel.org/security/lsm-development.html> (visited on 07/27/2026).
- [52] The Linux Kernel Documentation. *Linux Tracing Technologies*. 2026. URL: <https://docs.kernel.org/trace/index.html> (visited on 07/27/2026).

BIBLIOGRAPHY

- [53] The Linux Kernel Documentation. *Pathname Lookup*. 2026. URL: <https://docs.kernel.org/filesystems/path-lookup.html> (visited on 08/06/2026).
- [54] The Linux Kernel Documentation. *Perf Ring Buffer*. 2026. URL: https://docs.kernel.org/userspace-api/perf_ring_buffer.html (visited on 07/27/2026).
- [55] The Linux Kernel Documentation. *Program Types and ELF Sections*. 2026. URL: https://docs.kernel.org/bpf/libbpf/program_types.html (visited on 07/27/2026).
- [56] The Linux Kernel Documentation. *The Linux Kernel API*. Version 6.15.0. Documentation for the `security_file_open` Linux Security Module hook. URL: https://docs.kernel.org/6.15/core-api/kernel-api.html#c.security_file_open (visited on 08/04/2026).
- [57] The Linux Kernel Documentation. *Using the Linux Kernel Tracepoints*. 2026. URL: <https://docs.kernel.org/trace/tracepoints.html> (visited on 07/27/2026).
- [58] The Minikube Authors. *Docker Driver*. Official Minikube documentation. 2024. URL: <https://minikube.sigs.k8s.io/docs/drivers/docker/> (visited on 08/13/2026).
- [59] The Tetragon Authors. *Tetragon Enforcement*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/enforcement/> (visited on 07/27/2026).
- [60] The Tetragon Authors. *Tetragon Events*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/events/> (visited on 07/27/2026).
- [61] The Tetragon Authors. *Tetragon Installation FAQ*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/installation/faq/> (visited on 07/27/2026).
- [62] The Tetragon Authors. *Tetragon Tracing Policies*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/tracing-policy/> (visited on 07/27/2026).
- [63] The Tetragon Authors. *Tetragon Tracing Policy Selectors*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/tracing-policy/selectors/> (visited on 07/27/2026).
- [64] Uber Go. *Zap: Structured, Levelled Logging in Go*. URL: <https://github.com/uber-go/zap> (visited on 07/31/2026).

To my family and friends, for their support.